



Red Hat OpenShift AI Self-Managed 3.4

Working with Llama Stack

Working with Llama Stack in Red Hat OpenShift AI Self-Managed

Red Hat OpenShift AI Self-Managed 3.4 Working with Llama Stack

Working with Llama Stack in Red Hat OpenShift AI Self-Managed

Legal Notice

Copyright © Red Hat.

Except as otherwise noted below, the text of and illustrations in this documentation are licensed by Red Hat under the Creative Commons Attribution–Share Alike 3.0 Unported license . If you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, the Red Hat logo, JBoss, Hibernate, and RHCE are trademarks or registered trademarks of Red Hat, LLC. or its subsidiaries in the United States and other countries.

Linux[®] is the registered trademark of Linus Torvalds in the United States and other countries.

XFS is a trademark or registered trademark of Hewlett Packard Enterprise Development LP or its subsidiaries in the United States and other countries.

The OpenStack[®] Word Mark and OpenStack logo are trademarks or registered trademarks of the Linux Foundation, used under license.

All other trademarks are the property of their respective owners.

Abstract

As a cluster administrator, you can use the Llama Stack Operator in Red Hat OpenShift AI.

Table of Contents

CHAPTER 1. OVERVIEW OF LLAMA STACK	4
1.1. LLAMA STACK APIS	5
1.1.1. Supported Llama Stack APIs in OpenShift AI	5
1.1.1.1. Datasets_IO API	5
1.1.1.2. Evaluation API	5
1.1.1.3. Inference API	6
1.1.1.4. Safety API	6
1.1.1.5. Tool Runtime API	6
1.1.1.6. Vector_IO API	6
1.2. OPENAI-COMPATIBLE APIS IN LLAMA STACK	7
1.2.1. Supported OpenAI-compatible APIs in OpenShift AI	7
1.2.1.1. Models API	8
1.2.1.2. Chat Completions API	8
1.2.1.3. Completions API	9
1.2.1.4. Embeddings API	9
1.2.1.5. Files API	9
1.2.1.6. Vector Stores API	12
1.2.1.7. Vector Store Files API	12
1.2.1.8. Responses API	12
1.2.2. OpenAI compatibility for RAG APIs in Llama Stack	14
1.3. OPENAI-COMPATIBLE FILE CITATION ANNOTATIONS	15
1.3.1. OpenAI-compatible file citation annotations in Llama Stack	15
1.3.2. Viewing file citation annotations in Responses API output	16
1.3.3. File citation annotation reference	18
1.3.3.1. Annotation location	18
1.3.3.2. Supported annotation type	19
1.3.3.3. File citation fields	19
1.3.3.4. Annotation behavior	19
1.4. LLAMA STACK API PROVIDER SUPPORT	19
CHAPTER 2. ACTIVATING THE LLAMA STACK OPERATOR	23
CHAPTER 3. DEPLOYING A LLAMA STACK SERVER	25
CHAPTER 4. LLAMA STACK APPLICATION EXAMPLES	28
4.1. DEPLOYING A RAG STACK IN A PROJECT	28
4.1.1. Overview of RAG	29
4.1.1.1. Audience for RAG	29
4.1.2. Overview of vector databases	29
4.1.2.1. Overview of Milvus vector databases	30
4.1.2.2. Overview of Qdrant vector databases	32
4.1.2.3. Overview of FAISS vector databases	32
4.1.2.4. Overview of pgvector vector databases	33
4.1.3. Deploying a Llama model with KServe	34
4.1.4. Testing your vLLM model endpoints	38
4.1.5. Deploying a remote Milvus vector database	40
4.1.6. Deploying a LlamaStackDistribution instance	44
4.1.6.1. Example A: LlamaStackDistribution with Inline Milvus	47
4.1.6.2. Example B: LlamaStackDistribution with Remote Milvus	49
4.1.6.3. Example C: LlamaStackDistribution with Inline FAISS	51
4.1.6.4. Example D: LlamaStackDistribution with Remote PostgreSQL with pgvector	53
4.1.7. Ingesting content into a Llama model	56

4.1.8. Querying ingested content in a Llama model	60
4.1.9. Preparing documents with Docling for Llama Stack retrieval	64
4.1.10. About Llama stack search types	68
4.1.10.1. Supported search modes	68
4.1.10.1.1. Keyword search	68
4.1.10.1.2. Vector search	69
4.1.10.1.3. Hybrid search	69
4.2. EVALUATING RAG SYSTEMS WITH LLAMA STACK	70
4.2.1. Understanding RAG evaluation providers	70
4.2.2. Using Ragas with Llama Stack	70
4.2.3. Benchmarking embedding models with BEIR datasets and Llama Stack	71
4.2.3.1. BEIR benchmarking command-line options	74
4.3. USING POSTGRESQL IN LLAMA STACK	76
4.3.1. Understanding PostgreSQL in Llama Stack	76
4.3.1.1. Understanding Llama Stack metadata storage	76
4.3.1.1.1. Role of metadata storage in Llama Stack	76
4.3.1.1.2. PostgreSQL metadata storage backends	77
4.3.2. Deploying and Configuring PostgreSQL	77
4.3.2.1. Deploying a PostgreSQL instance with pgvector	77
4.3.2.2. Configuring the pgvector remote provider in Llama Stack	81
4.4. USING QDRANT IN LLAMA STACK	83
4.4.1. Overview of Qdrant vector databases	83
4.4.2. Deploying a Qdrant vector database	84
4.4.3. Configuring the Qdrant remote provider in Llama Stack	87
4.4.4. Performing vector operations with Qdrant	89
4.4.4.1. Add files to a vector store	91
4.4.4.2. Query a vector store	91
4.4.4.3. Delete a vector store	91
4.5. CONFIGURING LLAMA STACK WITH OAUTH AUTHENTICATION	91
4.6. CONFIGURING ATTRIBUTE-BASED ACCESS CONTROL (ABAC) ON YOUR LLAMA STACK SERVER	96
4.7. ABOUT USING SELF-SIGNED CERTIFICATES WITH LLAMA STACK	98
4.8. ENABLING HIGH AVAILABILITY AND AUTOSCALING FOR LLAMA STACK	99

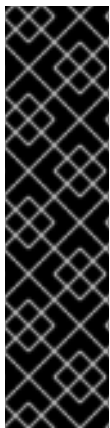
CHAPTER 1. OVERVIEW OF LLAMA STACK

Llama Stack is a unified AI runtime environment designed to simplify the deployment and management of generative AI workloads on OpenShift AI. In OpenShift, the Llama Stack Operator manages the deployment lifecycle of these components, ensuring scalability, consistency, and integration with OpenShift AI projects. Llama Stack integrates model inference, embedding generation, vector storage, and retrieval services into a single stack that is optimized for retrieval-augmented generation (RAG) and agent-based AI workflows.

Llama Stack concepts

- **Llama Stack Operator** Installs and manages Llama Stack server instances in OpenShift AI, handling lifecycle operations such as deployment, scaling, and updates.
- **The `run.yaml` file** Defines which APIs are enabled and how backend providers are configured for a Llama Stack server. Red Hat ships a default **`run.yaml`** that supports common deployment scenarios. You can provide a custom **`run.yaml`** to enable advanced workflows or integrate additional providers.
- **LlamaStackDistribution custom resource** Declares the runtime configuration for a Llama Stack server, including model providers, embedding configuration, vector storage, and persistence settings.

OpenShift AI ships with a Llama Stack Distribution that runs the Llama Stack server in a containerized environment. For the Llama Stack Operator version included in this release of OpenShift AI, see [Supported Configurations for 3.x](#).



IMPORTANT

Llama Stack integration is currently available in Red Hat OpenShift AI 3.4 as a Technology Preview feature. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production.

These features provide early access to upcoming product capabilities, enabling customers to test functionality and provide feedback during development.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

Llama Stack includes the following core components:

- **Integration with OpenShift AI** Uses the **LlamaStackDistribution** custom resource to simplify configuration and deployment of AI workloads.
- **Inference model connections** Acts as a proxy between Llama Stack APIs and model inference servers, such as vLLM deployments.
- **Embedding generation** Generates vector embeddings used for retrieval. In OpenShift AI 3.2, remote embedding models are the recommended and default option for production deployments. Inline embedding models remain available for development and testing scenarios.
- **Vector storage** Stores and indexes embeddings by using supported vector databases, such as Milvus or PostgreSQL with the pgvector extension.

- **Metadata persistence** Stores vector store metadata, file references, and configuration state. In OpenShift AI 3.2, PostgreSQL is the default backend for production-grade deployments.
- **Retrieval workflows** Manages ingestion, chunking, embedding, and similarity search to support RAG workflows.
- **Agentic workflows** Enables agent-based interactions through supported APIs, such as OpenAI-compatible Responses and Chat Completions.

For information about deploying Llama Stack in OpenShift AI, see [Deploying a RAG stack in a project](#) .



NOTE

The Llama Stack Operator is not currently supported on IBM Z platform.

Llama Stack is supported on IBM Power (**ppc64le**) with limited functionality:

- The GenAI playground is currently unavailable on the IBM Power architecture.
- Although **milvus-lite** and **PostgreSQL**, with the **pgvector** extension, are listed as supported vector stores, they are not currently available on the IBM Power **ppc64le** architecture.

Additional resources

- [Llama Stack demos repository](#)
- [Llama Stack Kubernetes Operator documentation](#)
- [Llama Stack documentation](#)

1.1. LLAMA STACK APIS

You can use the following APIs from Llama Stack for AI actions such as evaluation, scoring, and inference:

1.1.1. Supported Llama Stack APIs in OpenShift AI

1.1.1.1. Datasets_IO API

- **Endpoint:** `/v1alpha/datasetio`.
- **Providers:** All dataset_io backends deployed through OpenShift AI.
- **Support level:** Technology Preview.

The Dataset_IO API manages the input and output of datasets and their content.

1.1.1.2. Evaluation API

- **Endpoint:** `/v1alpha/eval`.
- **Providers:** All evaluation backends deployed through OpenShift AI.

- **Support level:** Developer Preview.

The Evaluation API defines an evaluation task for models and datasets

1.1.1.3. Inference API

- **Endpoint:** `/v1alpha/inference`.
- **Providers:** All inference backends deployed through OpenShift AI.
- **Support level:** Developer Preview.



WARNING

The majority of the Inference API is deprecated. The Inference providers use the Completions and Chat Completions APIs now.

The Inference API enables conversational, message-based interactions with models served by Llama Stack in OpenShift AI.

1.1.1.4. Safety API

- **Endpoint:** `/v1/safety`.
- **Providers:** All safety backends deployed through OpenShift AI.
- **Support level:** Technology Preview.

The Safety API detects and prevents harmful content in model inputs and outputs.

1.1.1.5. Tool Runtime API

- **Endpoint:** `/v1/tool-runtime`.
- **Providers:** All tool runtime backends deployed through OpenShift AI.
- **Support level:** Developer Preview.

The Tool Runtime API allows a model to dynamically call a tool at runtime.

1.1.1.6. Vector_IO API

- **Endpoint:** `/v1/vector-io`.
- **Providers:** All vector_io backends deployed through OpenShift AI.
- **Support level:** Developer Preview.

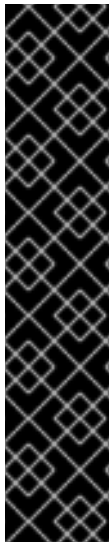
The Vector_IO API allows you to manage and query vector embeddings: numeric representations of data.

1.2. OPENAI-COMPATIBLE APIS IN LLAMA STACK

OpenShift AI includes a Llama Stack component that exposes OpenAI-compatible APIs. These APIs enable you to reuse existing OpenAI SDKs, tools, and workflows directly within your OpenShift environment, without changing your client code. This compatibility layer supports retrieval-augmented generation (RAG), inference, and embedding workloads by using OpenAI-compatible endpoints, schemas, and authentication patterns.

This compatibility layer has the following capabilities:

- **Standardized endpoints:** REST API paths align with OpenAI specifications.
- **Schema parity:** Request and response fields follow OpenAI data structures.



IMPORTANT

When connecting OpenAI SDKs or third-party tools to OpenShift AI, you must update the client configuration to use your deployment's Llama Stack route as the **base_url**.

When you use OpenAI-compatible SDKs, the **base_url must include the /v1 path suffix** so that requests are routed to the OpenAI-compatible API surface exposed by Llama Stack.

When you use OpenAI SDKs or send raw HTTP requests to Llama Stack, always include the **/v1** path suffix in the base URL.

For example: **http://llama-stack-service:8321/v1**

Using the service endpoint without **/v1** results in request failures.

These endpoints are exposed under the OpenAI compatibility layer and are distinct from the native Llama Stack APIs.

1.2.1. Supported OpenAI-compatible APIs in OpenShift AI

Before running the following examples, ensure you have:

- The OpenAI Python SDK installed: **pip install -q openai rich**
- A configured client pointing to your Llama Stack endpoint
- Model IDs from your deployment (see Models API section)

```
from openai import OpenAI
import rich
```

```
# We'll be using a llama-stack server deployed in {productname-short}.
# Once all pods associated to the LlamaStackDistribution are running,
# create the base_url using the llama-stack service hostname (with /v1 at the end when using openai
sdk)
```

```
base_url = "http://llama-stack-distribution-service.my-project.svc.cluster.local:8321/v1"
```

```
client = OpenAI(
```

```
    api_key="your-llama-stack-key",
    base_url=base_url
)
```

For more information, see [Deploying a Llama Stack server](#).

1.2.1.1. Models API

- **Endpoint:** `/v1/models`.
- **Providers:** All model-serving back ends configured within OpenShift AI.
- **Support level:** Technology Preview.

The Models API lists and retrieves available model resources from the Llama Stack deployment running on OpenShift AI. By using the Models API, you can enumerate models, view their capabilities, and verify deployment status through a standardized OpenAI-compatible interface.

Example code in Python:

```
# List models available in the llama-stack server
models = client.models.list()
rich.print(models)

# Select the first LLM and first embedding model
model_id = next(m for m in models if m.custom_metadata["model_type"] == "llm").id
embedding_model_id = (
    em := next(m for m in models if m.custom_metadata["model_type"] == "embedding")
).id
embedding_dimension = em.custom_metadata["embedding_dimension"]
```

1.2.1.2. Chat Completions API

- **Endpoint:** `/v1/chat/completions`.
- **Providers:** All inference back ends deployed through OpenShift AI.
- **Support level:** Technology Preview.

The Chat Completions API enables conversational, message-based interactions with models served by Llama Stack in OpenShift AI.

Example code in Python:

```
# Test chat completion functionality with a simple question
response = client.chat.completions.create(
    model=model_id,
    messages=[
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "What is the capital of France?"},
    ],
    temperature=0,
)
# Optional verification check
```

```
assert len(response.choices) > 0, "No response after basic inference on llama-stack server"
content = response.choices[0].message.content
rich.print(content)
```

1.2.1.3. Completions API

- **Endpoint:** `/v1/completions`.
- **Providers:** All inference back ends managed by OpenShift AI.
- **Support level:** Technology Preview.

The Completions API supports single-turn text generation and prompt completion.

Example code in Python:

```
# Test completion functionality with a simple question
response = client.completions.create(
    model=model_id,
    prompt="Answer with one word only: What is the capital of France?",
    max_tokens=64,
    temperature=0.1
)
# Optional verification check
assert len(response.choices) > 0, "No response after basic inference on llama-stack server"
content = response.choices[0].text
rich.print(content)
```

1.2.1.4. Embeddings API

- **Endpoint:** `/v1/embeddings`.
- **Providers:** All embedding models enabled in OpenShift AI.

The Embeddings API generates numerical embeddings for text or documents that can be used in downstream semantic search or RAG applications.

Example code in Python:

```
# Create text embeddings
response = client.embeddings.create(
    input="Your text string goes here",
    model=embedding_model_id
)
embedding = response.data[0].embedding
rich.print(embedding[:5] + ["..."] + embedding[-5:])
```

1.2.1.5. Files API

- **Endpoint:** `/v1/files`.
- **Providers:** File system-based file storage provider for managing files and documents stored locally in your cluster.

- **Support level:** Technology Preview.

The Files API manages file uploads for use in embedding and retrieval workflows.

Example code in Python:

```
import requests
from rich import print
from rich.rule import Rule
import time

# -----
# Download the PDF from url
# -----
print(Rule("[bold cyan]Downloading PDF[/bold cyan]"))

# We'll use IBM 2025-Q4 report to test RAG, as models don't have that info
pdf_url = "https://www.ibm.com/downloads/documents/us-en/1550f7eea8c0ded6"
filename = "ibm-Q4-2025-4q25-press-release.pdf"
title = "IBM-4Q25-Earnings-Press-Release"

print(" Fetching PDF from URL...")
response = requests.get(pdf_url)
response.raise_for_status()
print(" PDF fetched successfully")

print(f" Saving PDF as [bold]{filename}[/bold]...")
with open(filename, "wb") as f:
    f.write(response.content)
print(f" Downloaded and saved: [green]{filename}[/green]")

# -----
# Upload the PDF
# -----
print(Rule("[bold cyan]Uploading File[/bold cyan]"))

print(" 📁 Uploading file to Files API...")
with open(filename, "rb") as f:
    file_info = client.files.create(
        file=(filename, f),
        purpose="assistants"
    )

print(" File uploaded successfully")
print(file_info)

# -----
# Create vector store
# -----
print(Rule("[bold cyan]Creating Vector Store[/bold cyan]"))

provider_id = "milvus"

print(" Creating vector store with Milvus provider...")
vector_store = client.vector_stores.create(
    name="test_vector_store",
```

```

    extra_body={
        "embedding_model": embedding_model_id,
        "embedding_dimension": embedding_dimension,
        "provider_id": provider_id,
    },
)

print(" Vector store created")
print(vector_store)

# -----
# Add file to vector store
# -----
print(Rule("[bold cyan]Indexing File[/bold cyan]"))

print(" Adding uploaded file to vector store...")
vector_store_file = client.vector_stores.files.create(
    vector_store_id=vector_store.id,
    file_id=file_info.id,
    chunking_strategy={
        "type": "static",
        "static": {
            "max_chunk_size_tokens": 700,
            "chunk_overlap_tokens": 100,
        }
    },
    attributes={
        "title": title,
    },
)

print(" File added to vector store")
print(vector_store_file)

# -----
# Verify file is completed
# -----
print(Rule("[bold cyan]Waiting until file status is complete[/bold cyan]"))

# Wait for file processing to complete
print("Waiting for file processing to complete...")
max_wait_time = 300 # 5 minutes
start_time = time.time()

while time.time() - start_time < max_wait_time:
    files = client.vector_stores.files.list(vector_store_id=vector_store.id)
    if files.data:
        file_status = files.data[0].status
        print(f"File status: {file_status}")
        if file_status == "completed":
            print(" File processing completed!")
            break
        elif file_status == "failed":
            print("✗ File processing failed!")
            break

```

```

    time.sleep(5)
else:
    print("⚠ Timeout waiting for file processing")

# Verify file is completed
files = client.vector_stores.files.list(vector_store_id=vector_store.id)
if files.data:
    print(f"\nFinal file status: {files.data[0].status}")
    print(f"File details: {files.data[0]}")
else:
    print("No files found in vector store")

print(Rule("[bold green]All tasks completed successfully ✓[/bold green]"))

```

1.2.1.6. Vector Stores API

- **Endpoint:** `/v1/vector_stores/`.
- **Providers:** Inline and remote vector store providers configured in OpenShift AI.
- **Support level:** Technology Preview.

The Vector Stores API manages the creation, configuration, and lifecycle of vector store resources in Llama Stack. Through this API, you can create new vector stores, list existing ones, delete unused stores, and query their metadata, all using OpenAI-compatible request and response formats.

1.2.1.7. Vector Store Files API

- **Endpoint:** `/v1/vector_stores/{vector_store_id}/files`.
- **Providers:** Local inline provider configured for file storage and retrieval.
- **Support level:** Developer Preview.

The Vector Store Files API implements the OpenAI Vector Store Files interface and manages the association between document files and vector stores used for RAG workflows.

1.2.1.8. Responses API

- **Endpoint:** `/v1/responses`.
- **Providers:** All agents, inference, and vector providers configured in OpenShift AI.
- **Support level:** Developer Preview.

The Responses API generates model outputs by combining inference, file search, and tool-calling capabilities through a single OpenAI-compatible endpoint. It is particularly useful for retrieval-augmented generation (RAG) workflows that rely on the **file_search** tool to retrieve context from vector stores.

Example code in Python:

```

from rich import print
from rich.table import Table

```

```

system_instructions = """You are a financial document analysis assistant specialized in quarterly
earnings reports, annual filings, press releases, and earnings call transcripts.
You are designed to answer questions in a concise and professional manner.
Answer questions strictly using only the provided documents.
Base every answer strictly on the retrieved document content and cite the relevant section or excerpt
ID.
Do not use outside knowledge.
Do not guess, infer missing data, or fabricate numbers.
If the answer is not found in the retrieved content, reply: "I couldn't find relevant information in the
available files or my own knowledge."
Be concise, precise, and factual."""

```

```

examples = [
    {
        "input_query": "What do you know about IBM earnings in Q4, 2025? Summarize in one
sentence",
        "expected_answer": "IBM reported strong fourth-quarter results with revenue rising 12% to $19.7
billion, driven by double-digit growth in its Software and Infrastructure segments and a generative AI
book of business that has now surpassed $12.5 billion"
    },
    {
        "input_query": "What was the total value of IBM's generative AI book of business as reported in
the fourth quarter of 2025?",
        "expected_answer": "IBM reported that its generative AI book of business now stands at more
than $12.5 billion."
    },
    {
        "input_query": "What was IBM's reported free cash flow for the full year of 2025?",
        "expected_answer": (
            "IBM reported a full-year free cash flow of $14.7 billion, which was an increase of $2.0 billion
year-over-year"
        )
    },
    {
        "input_query": "How did the Software segment perform in terms of revenue during the fourth
quarter of 2025?",
        "expected_answer": (
            "The Software segment generated $9.0 billion in revenue, representing an increase of 14
percent (or 11 percent at constant currency)"
        )
    },
]

```

```

# Use the Responses API to create a results table comparing not using vs using
# the vector_store

```

```

table = Table(
    title="Answer Comparison (With vs Without Vector Store)",
    show_lines=True,
)

```

```

table.add_column("Question", style="cyan", no_wrap=False)
table.add_column("Expected Answer", style="magenta", no_wrap=False)
table.add_column("Answer (No Vector Store)", style="yellow", no_wrap=False)
table.add_column("Answer (With Vector Store)", style="green", no_wrap=False)

```

```

for example in examples:

```

```

question = example["input_query"]
expected_answer = example["expected_answer"]

# Ask question without vector_store
response_no_vs = client.responses.create(
    model=model_id,
    input=question,
    instructions=system_instructions,
)
answer_no_vs = response_no_vs.output_text.strip()

# Ask question with vector_store
response_vs = client.responses.create(
    model=model_id,
    input=question,
    instructions=system_instructions,
    tools=[
        {
            "type": "file_search",
            "vector_store_ids": [vector_store.id],
        }
    ],
)
answer_vs = response_vs.output_text.strip()

table.add_row(
    question,
    expected_answer,
    answer_no_vs,
    answer_vs,
)

# The table will take a while to be printed, as multiple queries to the responses API will be done
print(table)

```



NOTE

The Responses API is an experimental feature that is still under active development in OpenShift AI. While the API is already functional and suitable for evaluation, some endpoints and parameters remain under implementation and might change in future releases. This API is provided for testing and feedback purposes only and is not recommended for production use.

Additional resources

- [OpenAI API compatibility in Llama Stack](#)

1.2.2. OpenAI compatibility for RAG APIs in Llama Stack

OpenShift AI supports OpenAI-compatible request and response schemas for Llama Stack retrieval-augmented generation (RAG) workflows. This compatibility allows you to use OpenAI clients, tools, and schemas with Llama Stack for managing files, vector stores, and executing RAG queries through the Responses API.

OpenAI compatibility enables the following capabilities:

- You can use OpenAI SDKs and tools with Llama Stack by pointing the client to the Llama Stack OpenAI-compatible API path.
- You can manage files and vector stores by using OpenAI-compatible endpoints and invoke RAG workflows by using the Responses API with the **file_search** tool.

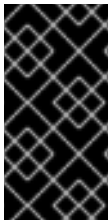
When configuring clients, the required **base_url** depends on the SDK that you use:

- **OpenAI SDKs** When you use an OpenAI-compatible SDK (for example, the OpenAI Python client), you must include the **/v1** path suffix in the base URL. For example:

```
`http://llama-stack-service:8321/v1`
```

- **Llama Stack SDK (llama_stack_client)** When you use the native Llama Stack SDK, set the base URL to the Llama Stack service endpoint without the **/v1** suffix. The SDK automatically appends the correct API paths. For example:

```
`http://llama-stack-service:8321`
```



IMPORTANT

When you use OpenAI-compatible SDKs or send raw HTTP requests to Llama Stack, always include the **/v1** path suffix in the base URL.

Using the service endpoint without **/v1** results in request failures.

Additional resources

- [OpenAI API compatibility in Llama Stack](#)
- [OpenAI API reference](#)
- [Llama Stack Python client API](#)

1.3. OPENAI-COMPATIBLE FILE CITATION ANNOTATIONS

Llama Stack supports OpenAI-compatible file citation annotations in Responses API outputs when using the **file_search** tool. These annotations enable applications to trace generated responses back to source documents without requiring changes to existing OpenAI client code.

1.3.1. OpenAI-compatible file citation annotations in Llama Stack

OpenShift AI provides OpenAI-compatible file citation annotations in Responses API outputs when using retrieval-augmented generation (RAG) with the **file_search** tool. These annotations enable applications to trace generated responses back to the source files used during retrieval without requiring changes to existing OpenAI client code. When you use the Responses API with the **file_search** tool, Llama Stack returns citation metadata that references the source file used to generate the response. Annotations are enabled by default.

Citation annotations have the following characteristics:

- They follow the same response structure defined by OpenAI.
- They appear in the **annotations** field of **output_text** response content.

- They identify the source file by ID and filename.
- They provide document-level attribution.

This feature improves transparency for RAG workflows while maintaining schema compatibility with OpenAI request and response formats.

In OpenShift AI, the following annotation capabilities are supported:

- Annotations are returned only through the Responses API.
- Annotations are returned only when using the **file_search** tool.
- The **file_citation** annotation type is supported.
- Attribution is provided at the document level.

Additional resources

- [OpenAI compatibility for RAG APIs in Llama Stack](#)
- [OpenAI-compatible APIs in Llama Stack](#)

1.3.2. Viewing file citation annotations in Responses API output

When you query ingested content by using the **file_search** tool with the Responses API, Llama Stack returns OpenAI-compatible **file_citation** annotations. These annotations identify the source files used during retrieval.

Prerequisites

- You have deployed a Llama Stack server.
- You have configured at least one inference model.
- You have created a vector store and ingested content into it.
- You can successfully execute a RAG query by using the **file_search** tool, as described in [Querying ingested content in a Llama model](#).
- You have access to a client environment, such as a Jupyter notebook or an OpenAI SDK client, that is correctly configured to send authenticated requests to the Llama Stack server.



NOTE

This procedure requires that content has already been ingested into a vector store. If no content is available, RAG queries return empty or non-contextual responses.

Procedure

1. In a Jupyter notebook cell or other configured client environment, run a RAG query by using the **file_search** tool.

```
response = client.responses.create(  
    model=model_id,
```

```

    input=query,
    instructions=system_instructions,
    tools=[
        {
            "type": "file_search",
            "vector_store_ids": [vector_store_id],
        }
    ],
)

```

2. Inspect the full response object rather than only the **output_text** property.

```
response.output
```

3. Access the **annotations** array.

```

annotations = response.output[0].content[0].annotations
print(annotations)

```

4. Review the **file_citation** annotation fields.

Example output:

```

[
  {
    "type": "file_citation",
    "file_id": "file-57610eaac6364459bfefae60377837b7",
    "filename": "redbankfinancial_about.pdf",
    "index": 139
  }
]

```

Each **file_citation** annotation includes the following fields:

- **file_id**: The identifier of the retrieved file.
- **filename**: The name of the source file.
- **index**: The index of the cited file in the list of files.

Multiple annotations can reference the same index position.

Optional: Using the OpenAI-compatible HTTP endpoint

If you use raw HTTP requests or an OpenAI SDK, send requests to the following endpoint:

/v1/responses

Ensure that your base URL includes the **/v1** path suffix, as described in [OpenAI compatibility for RAG APIs in Llama Stack](#).

**NOTE**

The accuracy and consistency of citation annotations depend on the capabilities of the underlying language model. Smaller or less capable models might produce less precise attributions, even when retrieval is functioning correctly. If citation results are incomplete or inconsistent, verify the model configuration and consider using a larger or more capable model.

Optional: Using the OpenAI-compatible endpoint

When you use an OpenAI SDK, configure the client **base_url** to include the **/v1** path suffix. The SDK automatically appends the appropriate endpoint path, such as **/responses**.

For example:

http://llama-stack-service:8321/v1

When you send raw HTTP requests, include both the **/v1** path suffix and the **/responses** endpoint in the full request URL.

For example:

http://llama-stack-service:8321/v1/responses

Ensure that **/v1** is included only once in the base URL. Do not append **/v1** multiple times.

For more information, see [OpenAI compatibility for RAG APIs in Llama Stack](#).

**NOTE**

The accuracy and consistency of citation annotations depend on the capabilities of the underlying language model. Smaller or less capable models might produce less precise attributions, even when retrieval is functioning correctly. If citation results are incomplete or inconsistent, verify the model configuration and consider using a larger or more capable model.

Verification

- The response includes an **annotations** array under **output[].content[]**.
- Each annotation has **"type": "file_citation"**.
- The **file_id** and **filename** correspond to files stored in the specified vector store.

1.3.3. File citation annotation reference

This reference describes the **file_citation** annotation type returned by Llama Stack through the OpenAI-compatible Responses API.

1.3.3.1. Annotation location

Annotations are returned in the **annotations** field of **output_text** content items within the **output[].content[]** structure of the Responses API response.

"output": [

```

{
  "content": [
    {
      "type": "output_text",
      "text": "Example generated response.",
      "annotations": [ ... ]
    }
  ]
}

```

1.3.3.2. Supported annotation type

In OpenShift AI, Llama Stack returns the **file_citation** annotation type when using the **file_search** tool.

URL citation annotations

The **url_citation** type is defined in the OpenAI schema but is not produced by Llama Stack in OpenShift AI 3.3.

1.3.3.3. File citation fields

The **file_citation** annotation includes the following fields:

Field	Type	Description
type	string	Always file_citation
file_id	string	Identifier of the source file used during retrieval
filename	string	Name of the source file
index	integer	Index of the cited file in the list of files.

1.3.3.4. Annotation behavior

- Attribution is provided at the document level.
- Multiple annotations can reference the same index position.
- Chunk-level and token-level attribution are not supported.
- Annotations follow the OpenAI response schema without modification.

1.4. LLAMA STACK API PROVIDER SUPPORT

You can use Llama Stack to enable various Provider APIs and providers in OpenShift AI. The following table lists the supported providers included in OpenShift AI, enablement environment variables, disconnected environment support, and its current support status.

**WARNING**

The support status of the Llama Stack API providers has shifted between Technology Preview and Developer Preview across OpenShift AI versions.

Provider API	Providers	How to Enable	Disconnected support	Support status
Agents	inline::meta-reference  NOTE The Responses API is accessible from the Agents provider API.	Enabled by default	Yes	Developer Preview
Dataset IO	inline::localfs	Enabled by default	Yes	Technology Preview
	remote::huggingface	Enabled by default	No	Technology Preview
Evaluation	inline::ragas	Set the EMBEDDING_MODEL environment variable	No	Technology Preview
	remote::ragas	See the "Configuring the Ragas remote provider for production" documentation	No	Technology Preview
	remote::lmeval	Enabled by default	No	Technology Preview
Files	inline::localfs	Enabled by default	No	Technology Preview
Inference	remote::vllm	Set the VLLM_URL environment variable	Yes	Technology Preview

Provider API	Providers	How to Enable	Disconnected support	Support status
	inline::sentence-transformers	Set the ENABLE_SENTENCE_TRANSFORMERS environment variable to "true"	Yes	Technology Preview
	remote::azure	Set the AZURE_API_KEY environment variable	No	Technology Preview
	remote::bedrock	Set the AWS_ACCESS_KEY_ID environment variable	No	Technology Preview
	remote::openai	Set the OPENAI_API_KEY environment variable	No	Technology Preview
	remote::vertexai	Set the VERTEX_AI_PROJECT environment variable	No	Technology Preview
	remote::watsonx	Set the WATSONX_API_KEY environment variable	No	Technology Preview
Safety	remote::trustyai_fms	Enabled by default	No	Technology Preview
Scoring	inline::basic	Enabled by default	No	Technology Preview
	inline::braintrust	Enabled by default	No	Technology Preview
	inline::llm-as-a-judge	Enabled by default	No	Technology Preview
Tool_Runtime	remote::model-context-protocol	Enabled by default	No	Developer Preview
	inline::rag-runtime	Enabled by default	No	Developer Preview

Provider API	Providers	How to Enable	Disconnected support	Support status
	remote::brave-search	Enabled by default	No	Developer Preview
	remote::tavily-search	Enabled by default	No	Developer Preview
Vector_IO	inline::faiss	Set the ENABLE_FAISS environment variable	No	Technology Preview
	inline::milvus	Set the ENABLE_INLINE_MILVUS environment variable to "true"	Yes	Technology Preview
	remote::milvus	Set the MILVUS_ENDPOINT environment variable	Yes	Technology Preview
	remote::pgvector	Set the ENABLE_PGVECTOR environment variable	Yes	Technology Preview
	remote::qdrant	Set the ENABLE_QDRANT environment variable	Yes	Technology Preview

CHAPTER 2. ACTIVATING THE LLAMA STACK OPERATOR

You can activate the Llama Stack Operator on your OpenShift cluster by setting its **managementState** to **Managed** in the OpenShift AI Operator **DataScienceCluster** custom resource (CR). This setting enables Llama-based model serving without reinstalling or directly editing Operator subscriptions. You can edit the CR in the OpenShift web console or by using the OpenShift CLI (**oc**).



NOTE

As an alternative to following the steps in this procedure, you can activate the Llama Stack Operator from the OpenShift CLI (**oc**) by running the following command:

```
$ oc patch datasciencecluster <name> --type=merge -p {"spec":{"components":{"llamastackoperator":{"managementState":"Managed"}}}}
```

Replace **<name>** with your **DataScienceCluster** name, for example, **default-dsc**.

Prerequisites

- You have installed OpenShift 4.19 or newer.
- You have cluster administrator privileges.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS
- You have installed the Red Hat OpenShift AI Operator on your cluster.
- You have a **DataScienceCluster** custom resource in your environment; the default is **default-dsc**.
- Your infrastructure supports GPU-enabled instance types, for example, **g4dn.xlarge** on AWS.
- You have enabled GPU support in OpenShift AI, including installing the Node Feature Discovery Operator and NVIDIA GPU Operator. For more information, see [Installing the Node Feature Discovery Operator](#) and [Enabling NVIDIA GPUs](#).
- You have created a **NodeFeatureDiscovery** resource instance on your cluster, as described in [Installing the Node Feature Discovery Operator and creating a NodeFeatureDiscovery instance](#) in the NVIDIA documentation.
- You have created a **ClusterPolicy** resource instance with default values on your cluster, as described in [Creating the ClusterPolicy instance](#) in the NVIDIA documentation.



NOTE

For IBM Power, **ppc64le** architectures, CPU-only deployments are fully supported.

Procedure

1. Log in to the OpenShift web console as a cluster administrator.
2. In the **Administrator** perspective, click **Ecosystem → Installed Operators**.
3. Click the **Red Hat OpenShift AI Operator** to open its details.
4. Click the **Data Science Cluster** tab.
5. On the **DataScienceClusters** page, click the **default-dsc** object.
6. Click the **YAML** tab.
An embedded YAML editor opens, displaying the configuration for the **DataScienceCluster** custom resource.
7. In the YAML editor, locate the **spec.components** section. If the **llamastackoperator** field does not exist, add it. Then, set the **managementState** field to **Managed**:

```
spec:
  components:
    llamastackoperator:
      managementState: Managed
```

8. Click **Save** to apply your changes.

Verification

After you activate the Llama Stack Operator, verify that it is running in your cluster:

1. In the OpenShift web console, click **Workloads → Pods**.
2. From the **Project** list, select the **redhat-ods-applications** namespace.
3. Confirm that a pod with the label **name=llama-stack-k8s-operator** is displayed and has a status of **Running**.

CHAPTER 3. DEPLOYING A LLAMA STACK SERVER

Llama Stack allows you to create and deploy a server that enables various APIs for accessing AI services in your OpenShift AI cluster. You can create a **LlamaStackDistribution** custom resource for your desired use cases. You are responsible for provisioning and managing the PostgreSQL instance. The PostgreSQL database can be deployed in-cluster or hosted externally, as long as it is reachable from the cluster network.

The included procedure provides an example **LlamaStackDistribution** CR that deploys a Llama Stack server that enables the following setup:

- A connection to a vLLM inference service with a **llama32-3b** model.
- A connection to a remote vector database.
- Allocated persistent storage.
- Orchestration endpoints.

Prerequisites

- You have installed OpenShift 4.19 or newer.
- You have logged in to Red Hat OpenShift AI.
- You have cluster administrator privileges for your OpenShift cluster.
- You have activated the Llama Stack Operator in your cluster.
- You have access to a PostgreSQL version 14 or later instance that is reachable from the OpenShift cluster network.
- You have PostgreSQL credentials for that instance that allow Llama Stack to create the database and tables.
- You know the PostgreSQL hostname and database port to use for the **POSTGRES_HOST** and **POSTGRES_PORT** environment variables.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS

Procedure

1. In the OpenShift web console, select **Administrator** → **Quick Create** () → **Import YAML**, and create a CR similar to the following example **llamastack-custom-distribution.yaml** file:

Example llamastack-custom-distribution.yaml

```
apiVersion: llamastack.io/v1alpha1
kind: LlamaStackDistribution
```

```

metadata:
  name: llamastack-custom-distribution
  namespace: <project-name> # Replace with your OpenShift project
spec:
  replicas: 1
  server:
    containerSpec:
      env:
        - name: VLLM_URL
          value: 'https://llama32-3b.llamastack.svc.cluster.local/v1'
        - name: INFERENCE_MODEL
          value: llama32-3b
        - name: VLLM_TLS_VERIFY
          value: 'false'
        - name: POSTGRES_HOST
          value: <postgres-host>
        - name: POSTGRES_PORT
          value: '<postgres-port>' # Default PostgreSQL port is 5432
        - name: POSTGRES_DB
          value: llamastack
        - name: POSTGRES_USER
          value: llamastack
        - name: POSTGRES_PASSWORD
          valueFrom:
            secretKeyRef:
              key: password
              name: postgres-secret ❶
      name: llama-stack
      port: 8321
    distribution:
      name: 'rh-dev'
    storage:
      size: 20Gi
      mountPath: <custom-mount-path> ## Defaults to /opt/app-root/src/.llama/distributions/rh/

```

- ❶ Create the secret in the same namespace as the **LlamaStackDistribution** resource. Avoid placing passwords directly on the command line, as they can be stored in shell history. Instead, create a file that contains only the database password and use that file to create the secret, or create the secret by using the OpenShift web console.

For example:

```

$ oc create secret generic postgres-secret --from-file=password=pg-password.txt -n
<project-name>
$ rm -f pg-password.txt

```

For more information about creating and managing Secrets, see [Providing sensitive data to pods by using secrets](#).

Ensure that the file **pg-password.txt** contains only the database password and is deleted after the secret is created.

Llama Stack automatically creates the metadata database specified by the **POSTGRES_DB** environment variable if it does not already exist, provided that the PostgreSQL user has sufficient privileges.

Verification

1. Check that the custom resource was created with the following command:

```
$ oc get llamastackdistribution -n llamastack
```

2. Check the running pods with the following command:

```
$ oc get pods -n llamastack | grep llamastack-custom-distribution
```

3. Check the logs with the following command:

```
$ oc logs -n llamastack -l app=llama-stack
```

Example output

```
INFO: Started server process
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://[':::', '0.0.0.0']:8321
```

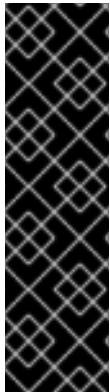
CHAPTER 4. LLAMA STACK APPLICATION EXAMPLES

Use the following examples to deploy and configure Llama Stack applications on OpenShift AI. These examples include deploying a RAG stack, evaluating RAG systems, and configuring authentication and availability options.

The following documentation includes example workflows:

- Deploying a RAG stack in a data science project
- Evaluating RAG systems with Llama Stack
- Using supported vector stores with Llama Stack
- Configuring Llama Stack with OAuth authentication

4.1. DEPLOYING A RAG STACK IN A PROJECT



IMPORTANT

This feature is currently available in Red Hat OpenShift AI 3.4 as a Technology Preview feature. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

As an OpenShift cluster administrator, you can deploy a Retrieval-Augmented Generation (RAG) stack in OpenShift AI. This stack provides the infrastructure, including LLM inference, vector storage, and retrieval services that data scientists and AI engineers use to build conversational workflows in their projects.

To deploy the RAG stack in a project, complete the following tasks:

- Activate the Llama Stack Operator in OpenShift AI.
- Enable GPU support on the OpenShift cluster. This task includes installing the required NVIDIA Operators.
- Deploy an inference model, for example, the llama-3.2-3b-instruct model. This task includes creating a storage connection and configuring GPU allocation.
- Create a **LlamaStackDistribution** instance to enable RAG functionality. This action deploys LlamaStack alongside a Milvus vector store and connects both components to the inference model.
- Ingest domain data into the configured vector store by running Docling in an AI pipeline or Jupyter notebook. This process keeps the embeddings synchronized with the source data.
- Expose and secure the model endpoints.

4.1.1. Overview of RAG

Retrieval-augmented generation (RAG) in OpenShift AI enhances large language models (LLMs) by integrating domain-specific data sources directly into the model's context. Domain-specific data sources can be structured data, such as relational database tables, or unstructured data, such as PDF documents.

RAG indexes content and builds an embedding store that data scientists and AI engineers can query. When data scientists or AI engineers pose a question to a RAG chatbot, the RAG pipeline retrieves the most relevant pieces of data, passes them to the LLM as context, and generates a response that reflects both the prompt and the retrieved content.

By implementing RAG, data scientists and AI engineers can obtain tailored, accurate, and verifiable answers to complex queries based on their own datasets within a project.

4.1.1.1. Audience for RAG

The target audience for RAG is practitioners who build data-grounded conversational AI applications using OpenShift AI infrastructure.

For Data Scientists

Data scientists can use RAG to prototype and validate models that answer natural-language queries against data sources without managing low-level embedding pipelines or vector stores. They can focus on creating prompts and evaluating model outputs instead of building retrieval infrastructure.

For MLOps Engineers

MLOps engineers typically deploy and operate RAG pipelines in production. Within OpenShift AI, they manage LLM endpoints, monitor performance, and ensure that both retrieval and generation scale reliably. RAG decouples vector store maintenance from the serving layer, enabling MLOps engineers to apply CI/CD workflows to data ingestion and model deployment alike.

For Data Engineers

Data engineers build workflows to load data into storage that OpenShift AI indexes. They keep embeddings in sync with source systems, such as S3 buckets or relational tables to ensure that chatbot responses are accurate.

For AI Engineers

AI engineers architect RAG chatbots by defining prompt templates, retrieval methods, and fallback logic. They configure agents and add domain-specific tools, such as OpenShift job triggers, enabling rapid iteration.

4.1.2. Overview of vector databases

Vector databases are a core component of retrieval-augmented generation (RAG) in OpenShift AI. They store and index vector embeddings that represent the semantic meaning of text or other data. When integrated with Llama Stack, vector databases enable applications to retrieve relevant context and combine it with large language model (LLM) inference.

Vector databases provide the following capabilities:

- Store vector embeddings generated by embedding models.
- Support efficient similarity search to retrieve semantically related content.
- Enable RAG workflows by supplying the LLM with contextually relevant data.

In OpenShift AI, vector databases are configured and managed through the Llama Stack Operator as part of a **LlamaStackDistribution**. PostgreSQL is the default and recommended metadata store for Llama Stack, supporting production-ready persistence, concurrency, and scalability.

The following vector database options are supported in OpenShift AI:

- **Inline Milvus** Inline Milvus runs embedded within the Llama Stack Distribution (LSD) pod and is suitable for development and small-scale RAG workloads. Inline Milvus uses PostgreSQL as the backing metadata store by default. This option provides a simplified deployment model while retaining durable metadata storage. The inline Milvus provider is not enabled by default. To enable inline Milvus, set the `ENABLE_INLINE_MILVUS` environment variable to **"true"** in the **LlamaStackDistribution**.
- **Inline FAISS** Inline FAISS uses the FAISS (Facebook AI Similarity Search) library to provide an in-process vector store for RAG workflows. Inline FAISS is designed for experimentation, prototyping, and development scenarios where simplicity and low operational overhead are priorities. Inline FAISS also relies on PostgreSQL for metadata storage.
- **Remote Milvus** Remote Milvus runs as a standalone vector database service, either within the cluster or as an external managed deployment. This option is suitable for large-scale or production-grade RAG workloads that require high availability, horizontal scalability, and isolation from the Llama Stack server. In OpenShift environments, Milvus typically requires an accompanying etcd service for coordination. For more information, see [Providing redundancy with etcd](#).
- **Remote PostgreSQL with pgvector** PostgreSQL with the pgvector extension provides a production-ready vector database option that integrates vector similarity search directly into PostgreSQL. This option is well suited for environments that already operate PostgreSQL and require durable storage, transactional consistency, and centralized management. pgvector enables Llama Stack to store embeddings and perform similarity search without deploying a separate vector database service.

Consider the following guidance when choosing a vector database for your RAG workloads:

- Use **Inline Milvus** or **Inline FAISS** for development, testing, or early experimentation.
- Use **Remote Milvus** when you require large-scale vector indexing and high-throughput similarity search.
- Use **PostgreSQL with pgvector** when you want production-ready persistence and integration with existing PostgreSQL-based data platforms.

SQLite-based storage is no longer recommended for production deployments. PostgreSQL-based backends provide improved reliability, concurrency, and scalability as Llama Stack moves toward general availability.

4.1.2.1. Overview of Milvus vector databases

Milvus is an open source vector database designed for high-performance similarity search across large volumes of embedding data. In OpenShift AI, Milvus is supported as a vector store provider for Llama Stack and enables retrieval-augmented generation (RAG) workloads that require efficient vector indexing, scalable search, and durable storage.

Production-grade Llama Stack deployments default to PostgreSQL for metadata persistence. When Milvus is used as the vector store, PostgreSQL is typically used for Llama Stack metadata, while Milvus manages vector indexes and similarity search.

Milvus vector databases provide the following capabilities in OpenShift AI:

- High-performance similarity search using Approximate Nearest Neighbor (ANN) algorithms
- Efficient indexing and query optimization for dense embeddings
- Persistent storage of vector data
- Integration with Llama Stack through an OpenAI-compatible Vector Stores API

In a typical RAG workflow in OpenShift AI, the following responsibilities are separated:

- **Embedding generation** Embeddings are generated by the configured embedding provider. Remote embedding models are the recommended and default option for production deployments.
- **Vector storage and retrieval** Milvus stores embedding vectors and performs similarity search operations.
- **Metadata persistence** Llama Stack stores vector store metadata, file references, and configuration state using PostgreSQL in production deployments.
- **Llama Stack server** Coordinates ingestion, retrieval, and model inference through a unified API surface.

In OpenShift AI, Milvus can be used in the following operational modes:

- **Inline Milvus Lite** Runs embedded within the Llama Stack Distribution pod. Inline Milvus Lite is intended for experimentation, development, or small datasets. It does not provide high availability or horizontal scalability and is not recommended for production use.



NOTE

The `ENABLE_INLINE_MILVUS` environment variable shown in the YAML below is required to enable the **inline::milvus** provider. Omitting it disables the inline Milvus vector store.

- **Remote Milvus** Runs as a standalone service within your OpenShift project or as an external managed Milvus deployment. Remote Milvus is recommended for production-grade RAG workloads.

A remote Milvus deployment typically includes the following components:

- A **Milvus service** that exposes a gRPC endpoint (port 19530) for client traffic
- An **etcd service** that Milvus uses for metadata coordination, collection state, and index management
- **Persistent storage** for durable vector data

Milvus requires a dedicated etcd instance for metadata coordination, even when running in standalone mode. Do not use the OpenShift control plane etcd for this purpose. For more information about etcd, see [Providing redundancy with etcd](#).

**IMPORTANT**

You must deploy a dedicated etcd service for Milvus or connect Milvus to an external etcd instance. Do not share the OpenShift control plane etcd with application workloads.

Use **Remote Milvus** when you require scalable vector search, high-performance retrieval, and integration with production-grade Llama Stack deployments in OpenShift AI.

For instructions on deploying Milvus as a remote vector database, see [Deploying a remote Milvus vector database](#).

4.1.2.2. Overview of Qdrant vector databases

Qdrant is an open source vector database optimized for high-performance similarity search and advanced filtering. In OpenShift AI, Qdrant is supported as a remote vector store provider for Llama Stack and can be used in retrieval-augmented generation (RAG) workloads that require efficient vector indexing and durable storage.

When used with Llama Stack in OpenShift AI, Qdrant provides:

- High-performance similarity search using Hierarchical Navigable Small World (HNSW) indexing
- Filtering based on stored metadata during vector search
- Persistent storage of vector data
- Integration through the OpenAI-compatible Vector Stores API

In a RAG workflow:

- Embeddings are generated by the configured embedding provider.
- Qdrant stores embedding vectors and performs similarity search.
- Llama Stack manages ingestion, retrieval, and model inference through a unified API.

In OpenShift AI, you must deploy Qdrant as a remote service, either within your OpenShift project or as an externally managed deployment.

**NOTE**

Inline Qdrant is not supported. To use Qdrant with Llama Stack in OpenShift AI, deploy Qdrant as a remote service.

A typical remote deployment includes:

- A Qdrant service exposing HTTP (port 6333) and gRPC (port 6334) endpoints
- Persistent storage for vector data
- Optional API key authentication

For deployment and configuration instructions, see [Using Qdrant in Llama Stack](#).

4.1.2.3. Overview of FAISS vector databases

The FAISS (Facebook AI Similarity Search) library is an open source framework for high-performance vector search and clustering. It is optimized for dense numerical embeddings and supports both CPU and GPU execution. In OpenShift AI, FAISS is supported as an **inline vector store provider** for Llama Stack, enabling fast, in-process similarity search without requiring a separate vector database service.

When you enable inline FAISS in a **LlamaStackDistribution**, Llama Stack uses FAISS as an embedded vector index that runs inside the Llama Stack server container. This configuration is designed for lightweight development, experimentation, and single-node retrieval-augmented generation (RAG) workflows. To enable inline FAISS, set the **ENABLE_FAISS** environment variable to **true** in the **LlamaStackDistribution** custom resource. For an example of deploying a LlamaStackDistribution instance with inline FAISS, see [Example C: LlamaStackDistribution with Inline FAISS](#).

Inline FAISS provides the following capabilities in OpenShift AI:

- In-process similarity search using FAISS indexes.
- Low-latency embedding ingestion and query operations.
- Simple deployment with no external vector database service.
- Compatibility with OpenAI-compatible Vector Stores API endpoints.

In OpenShift AI 3.2, inline FAISS relies on the Llama Stack metadata and persistence backend for managing vector store state. PostgreSQL is the default and recommended backend for production-grade deployments, even when FAISS is used as the inline vector index.

SQLite can be explicitly configured for local or on-the-fly development scenarios, but it is not recommended for production use.

Inline FAISS is suitable for the following use cases:

- Rapid prototyping of RAG workflows.
- Development or testing environments.
- Disconnected or single-node deployments where external vector databases are not required.



NOTE

Inline FAISS does not provide distributed storage, replication, or high availability. For production-grade RAG workloads that require durability, scalability, or multi-node access, use a remote vector database such as Milvus or PostgreSQL with the pgvector extension.

4.1.2.4. Overview of pgvector vector databases

pgvector is an open source PostgreSQL extension that enables vector similarity search on embedding data stored in relational tables. In OpenShift AI, PostgreSQL with the pgvector extension is supported as a remote vector database provider for the Llama Stack Operator. pgvector supports retrieval augmented generation workflows that require persistent vector storage while integrating with existing PostgreSQL environments.

pgvector vector databases provide the following capabilities in OpenShift AI:

- Storage of vector embeddings in PostgreSQL tables.
- Similarity search across embeddings by using pgvector distance metrics.

- Persistent storage of vectors alongside structured relational data.
- Integration with existing PostgreSQL security and operational tooling.

In a typical retrieval augmented generation workflow in OpenShift AI, your application uses the following components:

- **Inference provider** Generates embeddings and model responses.
- **Vector store provider** Stores embeddings and performs similarity search. When you use pgvector, PostgreSQL provides this capability as a remote vector store.
- **File storage provider** Stores the source files that are ingested into vector stores.
- **Llama Stack server** Provides a unified API surface, including an OpenAI compatible Vector Stores API.

When you ingest content, Llama Stack splits source material into chunks, generates embeddings, and stores them in PostgreSQL through the pgvector extension. When you query a vector store, Llama Stack performs similarity search and returns the most relevant chunks for use in prompts.

In OpenShift AI, pgvector is used in the following operational mode:

- **Remote PostgreSQL with pgvector**, which runs as a standalone PostgreSQL database service accessed by the Llama Stack server. This mode is suitable for development and production workloads that require persistent storage and integration with existing PostgreSQL infrastructure.

When you deploy PostgreSQL with the pgvector extension, you typically manage the following components:

- **Secrets** for PostgreSQL connection credentials.
- **Persistent storage** for durable database data.
- **A PostgreSQL service** that exposes a network endpoint.

PostgreSQL with pgvector does not require an external coordination service. Vector data, indexes, and metadata are stored directly in PostgreSQL tables and managed through standard database mechanisms.

Use PostgreSQL with pgvector when you require persistent vector storage and want to integrate vector search into existing PostgreSQL based data platforms within OpenShift AI. [Deploying a PostgreSQL instance with pgvector](#).

4.1.3. Deploying a Llama model with KServe

To use Llama Stack and retrieval-augmented generation (RAG) workloads in OpenShift AI, you must deploy a Llama model with a vLLM model server and configure KServe in KServe RawDeployment mode.



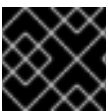
IMPORTANT

When deploying models using **KServe on IBM Power (ppc64le)**, ensure that you use only supported parameters for the model configuration.

Half (FP16) precision is not currently supported on this architecture. Attempting to use FP16 may result in a **NotImplementedError: "rotary_embedding_impl" not implemented for 'Half'** error.

Prerequisites

- You have installed OpenShift 4.19 or newer.
- You have logged in to Red Hat OpenShift AI.
- You have cluster administrator privileges for your OpenShift cluster.
- You have activated the Llama Stack Operator.
- You have installed KServe.
- You have enabled the model serving platform. For more information about enabling the model serving platform, see [Enabling the model serving platform](#).
- You can access the model serving platform in the dashboard configuration. For more information about setting dashboard configuration options, see [Customizing the dashboard](#).
- You have enabled GPU support in OpenShift AI, including installing the Node Feature Discovery Operator and NVIDIA GPU Operator. For more information, see [Installing the Node Feature Discovery Operator](#) and [Enabling NVIDIA GPUs](#).
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS
- You have created a project.
- The vLLM serving runtime is installed and available in your environment.
- You have created a storage connection for your model that contains a **URI - v1** connection type. This storage connection must define the location of your Llama 3.2 model artifacts. For example, **oci://quay.io/redhat-ai-services/modelcar-catalog:llama-3.2-3b-instruct**. For more information about creating storage connections, see [Adding a connection to your project](#).



PROCEDURE

These steps are only supported in OpenShift AI versions 2.19 and later.

1. In the OpenShift AI dashboard, navigate to the project details page and click the **Deployments** tab.
2. In the **Model serving platform** tile, click **Select model**.

3. Click the **Deploy model** button.
The **Deploy model** dialog opens.
4. Configure the deployment properties for your model:
 - a. In the **Model deployment name** field, enter a unique name for your deployment.
 - b. In the **Serving runtime** field, select **vLLM NVIDIA GPU serving runtime for KServe** from the drop-down list.
 - c. In the **Deployment mode** field, select **KServe RawDeployment** from the drop-down list.
 - d. Set **Number of model server replicas to deploy** to **1**.
 - e. In the **Model server size** field, select **Custom** from the drop-down list.
 - Set **CPUs requested** to **1 core**.
 - Set **Memory requested** to **10 GiB**.
 - Set **CPU limit** to **2 core**.
 - Set **Memory limit** to **14 GiB**.
 - Set **Accelerator** to **NVIDIA GPUs**.
 - Set **Accelerator count** to **1**.
 - f. From the **Connection type**, select a relevant data connection from the drop-down list.
5. In the **Additional serving runtime arguments** field, specify the following recommended arguments:

```
--dtype=half
--max-model-len=20000
--gpu-memory-utilization=0.95
--enable-chunked-prefill
--enable-auto-tool-choice
--tool-call-parser=llama3_json
--chat-template=/app/data/template/tool_chat_template_llama3.2_json.jinja
```

- a. Click **Deploy**.



NOTE

Model deployment can take several minutes, especially for the first model that is deployed on the cluster. Initial deployment may take more than 10 minutes while the relevant images download.

Verification

1. Verify that the **kserve-controller-manager** and **odh-model-controller** pods are running:
 - a. Open a new terminal window.
 - b. Log in to your OpenShift cluster from the CLI:

- c. In the upper-right corner of the OpenShift web console, click your user name and select **Copy login command**.
- d. After you have logged in, click **Display token**.
- e. Copy the **Log in with this token** command and paste it in the OpenShift CLI (**oc**).

```
$ oc login --token=<token> --server=<openshift_cluster_url>
```

- f. Enter the following command to verify that the **kserve-controller-manager** and **odh-model-controller** pods are running:

```
$ oc get pods -n redhat-ods-applications | grep -E 'kserve-controller-manager|odh-model-controller'
```

- g. Confirm that you see output similar to the following example:

```
kserve-controller-manager-7c865c9c9f-xyz12 1/1 Running 0 4m21s
odh-model-controller-7b7d5fd9cc-wxy34 1/1 Running 0 3m55s
```

- h. If you do not see either of the **kserve-controller-manager** and **odh-model-controller** pods, there could be a problem with your deployment. In addition, if the pods appear in the list, but their **Status** is not set to **Running**, check the pod logs for errors:

```
$ oc logs <pod-name> -n redhat-ods-applications
```

- i. Check the status of the inference service:

```
$ oc get inferencesservice -n llamastack
$ oc get pods -n <project name> | grep llama
```

- The deployment automatically creates the following resources:
 - A **ServingRuntime** resource.
 - An **InferenceService** resource, a **Deployment**, a pod, and a service pointing to the pod.
- Verify that the server is running. For example:

```
$ oc logs llama-32-3b-instruct-predictor-77f6574f76-8nl4r -n <project name>
```

Check for output similar to the following example log:

```
INFO 2025-05-15 11:23:52,750 __main__:498 server: Listening on [':', '0.0.0.0']:8321
INFO: Started server process [1]
INFO: Waiting for application startup.
INFO 2025-05-15 11:23:52,765 __main__:151 server: Starting up
INFO: Application startup complete.
INFO: Uvicorn running on http://[':', '0.0.0.0']:8321 (Press CTRL+C to quit)
```

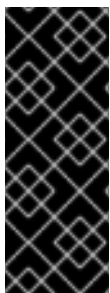
- The deployed model displays in the **Deployments** tab on the project details page for the project it was deployed under.
2. If you see a **ConvertTritonGPToLLVM** error in the pod logs when querying the **/v1/chat/completions** API, and the vLLM server restarts or returns a **500 Internal Server** error, apply the following workaround:
Before deploying the model, remove the **--enable-chunked-prefill** argument from the **Additional serving runtime arguments** field in the deployment dialog.

The error is displayed similar to the following:

```
/opt/vllm/lib64/python3.12/site-packages/vllm/attention/ops/prefix_prefill.py:36:0: error:
Failures have been detected while processing an MLIR pass pipeline
/opt/vllm/lib64/python3.12/site-packages/vllm/attention/ops/prefix_prefill.py:36:0: note:
Pipeline failed while executing ['ConvertTritonGPToLLVM' on 'builtin.module' operation]:
reproducer generated at `std::errs, please share the reproducer above with Triton project.`
INFO: 10.129.2.8:0 - "POST /v1/chat/completions HTTP/1.1" 500 Internal Server Error
```

4.1.4. Testing your vLLM model endpoints

To verify that your deployed Llama 3.2 model is accessible externally, ensure that your vLLM model server is exposed as a network endpoint. You can then test access to the model from outside both the OpenShift cluster and the OpenShift AI interface.



IMPORTANT

If you selected **Make deployed models available through an external route** during deployment, your vLLM model endpoint is already accessible outside the cluster. You do not need to manually expose the model server. Manually exposing vLLM model endpoints, for example, by using **oc expose**, creates an unsecured route unless you configure authentication. Avoid exposing endpoints without security controls to prevent unauthorized access.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have logged in to Red Hat OpenShift AI.
- You have activated the Llama Stack Operator in OpenShift AI.
- You have deployed an inference model, for example, the llama-3.2-3b-instruct model.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS

Procedure

1. Open a new terminal window.
 - a. Log in to your OpenShift cluster from the CLI:

- b. In the upper-right corner of the OpenShift web console, click your user name and select **Copy login command**.
- c. After you have logged in, click **Display token**.
- d. Copy the **Log in with this token** command and paste it in the OpenShift CLI (**oc**).

```
$ oc login --token=<token> --server=<openshift_cluster_url>
```

2. If you enabled **Require token authentication** during model deployment, retrieve your token:

```
$ export MODEL_TOKEN=$(oc get secret default-name-llama-32-3b-instruct-sa -n <project name> --template={{ .data.token }} | base64 -d)
```

3. Obtain your model endpoint URL:

- If you enabled **Make deployed models available through an external route** during model deployment, click **Endpoint details** on the **Deployments** page in the OpenShift AI dashboard to obtain your model endpoint URL.
- In addition, if you did not enable **Require token authentication** during model deployment, you can also enter the following command to retrieve the endpoint URL:

```
$ export MODEL_ENDPOINT="https://$(oc get route llama-32-3b-instruct -n <project name> --template={{ .spec.host }})"
```

4. Test the endpoint with a sample chat completion request:

- If you did not enable **Require token authentication** during model deployment, enter a chat completion request. For example:

```
$ curl -X POST $MODEL_ENDPOINT/v1/chat/completions \
-H "Content-Type: application/json" \
-d '{
  "model": "llama-32-3b-instruct",
  "messages": [
    {
      "role": "user",
      "content": "Hello"
    }
  ]
}'
```

- If you enabled **Require token authentication** during model deployment, include a token in your request. For example:

```
curl -s -k $MODEL_ENDPOINT/v1/chat/completions \
--header "Authorization: Bearer $MODEL_TOKEN" \
--header 'Content-Type: application/json' \
-d '{
  "model": "llama-32-3b-instruct",
  "messages": [
    {
      "role": "user",
      "content": "can you tell me a funny joke?"
    }
  ]
}'
```

```
}
]
}' | jq .
```



NOTE

The **-k** flag disables SSL verification and should only be used in test environments or with self-signed certificates.

Verification

Confirm that you received a JSON response containing a chat completion. For example:

```
{
  "id": "chatcmpl-05d24b91b08a4b78b0e084d4cc91dd7e",
  "object": "chat.completion",
  "created": 1747279170,
  "model": "llama-32-3b-instruct",
  "choices": [{
    "index": 0,
    "message": {
      "role": "assistant",
      "reasoning_content": null,
      "content": "Hello! It's nice to meet you. Is there something I can help you with or would you like to chat?",
      "tool_calls": []
    },
    "logprobs": null,
    "finish_reason": "stop",
    "stop_reason": null
  }],
  "usage": {
    "prompt_tokens": 37,
    "total_tokens": 62,
    "completion_tokens": 25,
    "prompt_tokens_details": null
  },
  "prompt_logprobs": null
}
```

If you do not receive a response similar to the example, verify that the endpoint URL and token are correct, and ensure your model deployment is running.

4.1.5. Deploying a remote Milvus vector database

To use Milvus as a remote vector database provider for Llama Stack in OpenShift AI, you must deploy Milvus and its required etcd service in your OpenShift project. This procedure shows how to deploy Milvus in standalone mode without the Milvus Operator.



NOTE

The following example configuration is intended for testing or evaluation environments. For production-grade deployments, see <https://milvus.io/docs> in the Milvus documentation.

Prerequisites

- You have installed OpenShift 4.19 or newer.
- You have enabled GPU support in OpenShift AI. This includes installing the Node Feature Discovery operator and NVIDIA GPU Operators. For more information, see [Installing the Node Feature Discovery operator](#) and [Enabling NVIDIA GPUs](#).
- You have cluster administrator privileges for your OpenShift cluster.
- You are logged in to Red Hat OpenShift AI.
- You have a StorageClass available that can provision persistent volumes.
- You created a root password to secure your Milvus service.
- You have deployed an inference model with vLLM, for example, the llama-3.2-3b-instruct model, and you have selected **Make deployed models available through an external route** and **Require token authentication** during model deployment.
- You have the correct inference model identifier, for example, llama-3-2-3b.
- You have the model endpoint URL, ending with **/v1**, such as **https://llama-32-3b-instruct-predictor:8443/v1**.
- You have the API token required to access the model endpoint.
- You have installed the OpenShift command line interface (**oc**) as described in [Installing the OpenShift CLI](#).

Procedure

1. In the OpenShift console, click the **Quick Create** () icon and then click the **Import YAML** option.
2. Verify that your project is the selected project.
3. In the **Import YAML** editor, paste the following manifest and click **Create**:

```
apiVersion: v1
kind: Secret
metadata:
  name: milvus-secret
type: Opaque
stringData:
  root-password: "MyStr0ngP@ssw0rd"
---
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: milvus-pvc
spec:
  accessModes:
    - ReadWriteOnce
resources:
  requests:
```

```
    storage: 20Gi
    volumeMode: Filesystem
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: etcd-deployment
  labels:
    app: etcd
spec:
  replicas: 1
  selector:
    matchLabels:
      app: etcd
  strategy:
    type: Recreate
  template:
    metadata:
      labels:
        app: etcd
    spec:
      containers:
        - name: etcd
          image: quay.io/coreos/etcd:v3.5.5
          command:
            - etcd
            - --advertise-client-urls=http://127.0.0.1:2379
            - --listen-client-urls=http://0.0.0.0:2379
            - --data-dir=/etcd
          ports:
            - containerPort: 2379
          volumeMounts:
            - name: etcd-data
              mountPath: /etcd
          env:
            - name: ETCD_AUTO_COMPACTION_MODE
              value: revision
            - name: ETCD_AUTO_COMPACTION_RETENTION
              value: "1000"
            - name: ETCD_QUOTA_BACKEND_BYTES
              value: "4294967296"
            - name: ETCD_SNAPSHOT_COUNT
              value: "50000"
      volumes:
        - name: etcd-data
          emptyDir: {}
      restartPolicy: Always
---
apiVersion: v1
kind: Service
metadata:
  name: etcd-service
spec:
  ports:
    - port: 2379
      targetPort: 2379
```

```

    selector:
      app: etcd
  ---
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    app: milvus-standalone
  name: milvus-standalone
spec:
  replicas: 1
  selector:
    matchLabels:
      app: milvus-standalone
  strategy:
    type: Recreate
  template:
    metadata:
      labels:
        app: milvus-standalone
    spec:
      containers:
        - name: milvus-standalone
          image: milvusdb/milvus:v2.6.0
          args: ["milvus", "run", "standalone"]
          env:
            - name: DEPLOY_MODE
              value: standalone
            - name: ETCD_ENDPOINTS
              value: etcd-service:2379
            - name: COMMON_STORAGETYPE
              value: local
            - name: MILVUS_ROOT_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: milvus-secret
                  key: root-password
          livenessProbe:
            exec:
              command: ["curl", "-f", "http://localhost:9091/healthz"]
            initialDelaySeconds: 90
            periodSeconds: 30
            timeoutSeconds: 20
            failureThreshold: 5
          ports:
            - containerPort: 19530
              protocol: TCP
            - containerPort: 9091
              protocol: TCP
          volumeMounts:
            - name: milvus-data
              mountPath: /var/lib/milvus
      restartPolicy: Always
      volumes:
        - name: milvus-data
          persistentVolumeClaim:

```

```

      claimName: milvus-pvc
---
apiVersion: v1
kind: Service
metadata:
  name: milvus-service
spec:
  selector:
    app: milvus-standalone
  ports:
    - name: grpc
      port: 19530
      targetPort: 19530
    - name: http
      port: 9091
      targetPort: 9091

```



NOTE

- Use the gRPC port (**19530**) for the **MILVUS_ENDPOINT** setting in Llama Stack.
- The HTTP port (**9091**) is reserved for health checks.
- If you deploy Milvus in a different namespace, use the fully qualified service name in your Llama Stack configuration. For example: **http://milvus-service.<namespace>.svc.cluster.local:19530**

Verification

1. In the OpenShift web console, click **Workloads → Deployments**.
2. Verify that both **etcd-deployment** and **milvus-standalone** show a status of **1 of 1 pods available**.
3. Click **Pods** in the navigation panel and confirm that pods for both deployments are **Running**.
4. Click the **milvus-standalone** pod name, then select the **Logs** tab.
5. Verify that Milvus reports a healthy startup with output similar to:

```

Milvus Standalone is ready to serve ...
Listening on 0.0.0.0:19530 (gRPC)

```

6. Click **Networking → Services** and confirm that the **milvus-service** and **etcd-service** resources exist and are exposed on ports **19530** and **2379**, respectively.
7. (Optional) Click **Pods → milvus-standalone → Terminal** and run the following health check:

```
curl http://localhost:9091/healthz
```

A response of **{"status": "healthy"}** confirms that Milvus is running correctly.

4.1.6. Deploying a LlamaStackDistribution instance

You can deploy Llama Stack with retrieval-augmented generation (RAG) by pairing it with a vLLM-served Llama 3.2 model. This module provides the following deployment examples of the **LlamaStackDistribution** custom resource (CR):

- **Example A:** Inline Milvus (embedded, single-node, remote embeddings)
- **Example B:** Remote Milvus (external service, inline embeddings served with the sentence-transformers library)
- **Example C:** Inline FAISS (embedded, single node, inline embeddings served with the sentence-transformers library)
- **Example D:** Remote PostgreSQL with pgvector (external service, remote embeddings)

Prerequisites

- You have installed OpenShift 4.19 or newer.
- You have enabled GPU support in OpenShift AI. This includes installing the Node Feature Discovery Operator and NVIDIA GPU Operator. For more information, see [Installing the Node Feature Discovery Operator](#) and [Enabling NVIDIA GPUs](#).
- You have cluster administrator privileges for your OpenShift cluster.
- You are logged in to Red Hat OpenShift AI.
- You have activated the Llama Stack Operator in OpenShift AI.
- You have deployed an inference model with vLLM (for example, **llama-3.2-3b-instruct**) and selected **Make deployed models available through an external route** and **Require token authentication** during model deployment. In addition, in **Add custom runtime arguments**, you have added **--enable-auto-tool-choice**.
- You have the correct inference model identifier, for example, **llama-3-2-3b**.
- You have the model endpoint URL ending with **/v1**, for example, **https://llama-32-3b-instruct-predictor:8443/v1**.
- You have the API token required to access the model endpoint.
- You have installed the PostgreSQL Operator version 14 or later and configured a PostgreSQL database for Llama Stack metadata storage. For more information, see the documentation for "Deploying a Llama Stack server".
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS

Procedure

1. Open a new terminal window and log in to your OpenShift cluster from the CLI:
In the upper-right corner of the OpenShift web console, click your user name and select **Copy login command**. After you have logged in, click **Display token**. Copy the **Log in with this token** command and paste it in the OpenShift CLI (**oc**).

```
$ oc login --token=<token> --server=<openshift_cluster_url>
```

2. Create a secret that contains the inference model and the remote embeddings environment variables:

```
# Remote LLM
export INFERENCE_MODEL="llama-3-2-3b"
export VLLM_URL="https://llama-32-3b-instruct-predictor:8443/v1"
export VLLM_TLS_VERIFY="false" # Use "true" in production
export VLLM_API_TOKEN="<token identifier>"
export VLLM_MAX_TOKENS=16384

# Remote embedding configuration
export EMBEDDING_MODEL="nomic-embed-text-v1-5"
export EMBEDDING_PROVIDER_MODEL_ID="nomic-embed-text-v1-5"
export VLLM_EMBEDDING_URL="<embedding-endpoint>/v1"
export VLLM_EMBEDDING_API_TOKEN="<embedding-token>"
export VLLM_EMBEDDING_MAX_TOKENS=8192
export VLLM_EMBEDDING_TLS_VERIFY="true"

oc create secret generic llama-stack-secret -n <project-name> \
  --from-literal=INFERENCE_MODEL="$INFERENCE_MODEL" \
  --from-literal=VLLM_URL="$VLLM_URL" \
  --from-literal=VLLM_TLS_VERIFY="$VLLM_TLS_VERIFY" \
  --from-literal=VLLM_API_TOKEN="$VLLM_API_TOKEN" \
  --from-literal=VLLM_MAX_TOKENS="$VLLM_MAX_TOKENS" \
  --from-literal=EMBEDDING_MODEL="$EMBEDDING_MODEL" \
  --from-
literal=EMBEDDING_PROVIDER_MODEL_ID="$EMBEDDING_PROVIDER_MODEL_ID" \
  --from-literal=VLLM_EMBEDDING_URL="$VLLM_EMBEDDING_URL" \
  --from-literal=VLLM_EMBEDDING_TLS_VERIFY="$VLLM_EMBEDDING_TLS_VERIFY" \
  --from-literal=VLLM_EMBEDDING_API_TOKEN="$VLLM_EMBEDDING_API_TOKEN" \
  --from-
literal=VLLM_EMBEDDING_MAX_TOKENS="$VLLM_EMBEDDING_MAX_TOKENS"
```

3. Choose **one** of the following deployment examples:

IMPORTANT

To use inline embeddings served with the **sentence-transformers** library, as displayed in Examples B and C, you must explicitly enable the provider by setting the following environment variables on the **LlamaStackDistribution** custom resource:

```
- name: ENABLE_SENTENCE_TRANSFORMERS
  value: "true"
- name: EMBEDDING_PROVIDER
  value: "sentence-transformers"
```

Starting in OpenShift AI 3.4, the **inline::sentence-transformers** provider is no longer enabled by default. If **ENABLE_SENTENCE_TRANSFORMERS** is not set to **"true"**, inline embeddings are unavailable.

IMPORTANT

In a disconnected environment, after enabling the **inline::sentence-transformers** provider, add the following parameters to your **LlamaStackDistribution** custom resource so that model weights are resolved from the local cache:

```
- name: SENTENCE_TRANSFORMERS_HOME
  value: /opt/app-root/src/.cache/huggingface/hub
- name: HF_HUB_OFFLINE
  value: "1"
- name: TRANSFORMERS_OFFLINE
  value: "1"
- name: HF_DATASETS_OFFLINE
  value: "1"
```

The built-in Llama Stack tool **websearch** is not available in the Llama Stack Distribution in disconnected environments. In addition, the built-in Llama Stack tool **wolfram_alpha** is not available in the Llama Stack Distribution in all clusters.

4.1.6.1. Example A: LlamaStackDistribution with Inline Milvus

Use this example for development or small datasets where an embedded, single-node Milvus is sufficient. This example uses remote embeddings.

NOTE

The `ENABLE_INLINE_MILVUS` environment variable shown in the YAML below is required to enable the **inline::milvus** provider. Omitting it disables the inline Milvus vector store.

1. In the OpenShift web console, select **Administrator** → **Quick Create** () → **Import YAML**, and create a CR similar to the following:

```
apiVersion: llamastack.io/v1alpha1
kind: LlamaStackDistribution
metadata:
  name: lsd-llama-milvus-inline
spec:
  replicas: 1
  server:
    containerSpec:
      resources:
        requests:
          cpu: "250m"
          memory: "500Mi"
        limits:
          cpu: 4
          memory: "12Gi"
  env:
    # PostgreSQL metadata store (required in {productname-short} 3.2)
    - name: POSTGRES_HOST
      value: <postgres-host>
    - name: POSTGRES_PORT
```

```

    value: "5432"
- name: POSTGRES_DB
  value: <postgres-database>
- name: POSTGRES_USER
  value: <postgres-username>
- name: POSTGRES_PASSWORD
  valueFrom:
    secretKeyRef:
      name: <postgres-secret-name>
      key: <postgres-password-key>
# Enable inline Milvus vector store
- name: ENABLE_INLINE_MILVUS
  value: "true"
# Remote LLM configuration
- name: INFERENCE_MODEL
  valueFrom:
    secretKeyRef:
      name: llama-stack-secret
      key: INFERENCE_MODEL
- name: VLLM_URL
  valueFrom:
    secretKeyRef:
      name: llama-stack-secret
      key: VLLM_URL
- name: VLLM_TLS_VERIFY
  valueFrom:
    secretKeyRef:
      name: llama-stack-secret
      key: VLLM_TLS_VERIFY
- name: VLLM_API_TOKEN
  valueFrom:
    secretKeyRef:
      name: llama-stack-secret
      key: VLLM_API_TOKEN
- name: VLLM_MAX_TOKENS
  valueFrom:
    secretKeyRef:
      name: llama-stack-secret
      key: VLLM_MAX_TOKENS

# Remote embedding configuration
- name: EMBEDDING_MODEL
  valueFrom:
    secretKeyRef:
      name: llama-stack-secret
      key: EMBEDDING_MODEL
- name: EMBEDDING_PROVIDER_MODEL_ID
  valueFrom:
    secretKeyRef:
      name: llama-stack-secret
      key: EMBEDDING_PROVIDER_MODEL_ID
- name: VLLM_EMBEDDING_URL
  valueFrom:
    secretKeyRef:
      name: llama-stack-secret
      key: VLLM_EMBEDDING_URL

```

```

- name: VLLM_EMBEDDING_TLS_VERIFY
  valueFrom:
    secretKeyRef:
      name: llama-stack-secret
      key: VLLM_EMBEDDING_TLS_VERIFY
- name: VLLM_EMBEDDING_API_TOKEN
  valueFrom:
    secretKeyRef:
      name: llama-stack-secret
      key: VLLM_EMBEDDING_API_TOKEN
- name: VLLM_EMBEDDING_MAX_TOKENS
  valueFrom:
    secretKeyRef:
      name: llama-stack-secret
      key: VLLM_EMBEDDING_MAX_TOKENS

- name: FMS_ORCHESTRATOR_URL
  value: "http://localhost"
name: llama-stack
port: 8321
distribution:
  name: rh-dev
storage:
  size: 5Gi

```



NOTE

The **rh-dev** value is an internal image reference. When you create the **LlamaStackDistribution** custom resource, the OpenShift AI Operator automatically resolves **rh-dev** to the container image in the appropriate registry. This internal image reference allows the underlying image to update without requiring changes to your custom resource.

4.1.6.2. Example B: LlamaStackDistribution with Remote Milvus

Use this example for production-grade or large datasets with an external Milvus service. This example uses inline embeddings served with the sentence-transformers library.



NOTE

The **ENABLE_SENTENCE_TRANSFORMERS** and **EMBEDDING_PROVIDER** environment variables shown in the YAML below are required to enable the **inline::sentence-transformers** provider in OpenShift AI 3.4. Omitting them disables inline embeddings.

1. Create the Milvus connection secret:

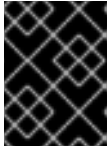
```

# Required: gRPC endpoint on port 19530
export MILVUS_ENDPOINT="tcp://milvus-service:19530"
export MILVUS_TOKEN="<milvus-root-or-user-token>"
export MILVUS_CONSISTENCY_LEVEL="Bounded" # Optional; choose per your
deployment

oc create secret generic milvus-secret \

```

```
--from-literal=MILVUS_ENDPOINT="$MILVUS_ENDPOINT" \
--from-literal=MILVUS_TOKEN="$MILVUS_TOKEN" \
--from-literal=MILVUS_CONSISTENCY_LEVEL="$MILVUS_CONSISTENCY_LEVEL"
```



IMPORTANT

Use the **gRPC port 19530** for **MILVUS_ENDPOINT**. Ports such as **9091** are typically used for health checks and are not valid for client traffic.

2. In the OpenShift web console, select **Administrator** → **Quick Create** () → **Import YAML**, and create a CR similar to the following:

```
apiVersion: llamastack.io/v1alpha1
kind: LlamaStackDistribution
metadata:
  name: lsd-llama-milvus-remote
spec:
  replicas: 1
  server:
    containerSpec:
      resources:
        requests:
          cpu: "250m"
          memory: "500Mi"
        limits:
          cpu: 4
          memory: "12Gi"
    env:
      # PostgreSQL metadata store (required in {productname-short} 3.2)
      - name: POSTGRES_HOST
        value: <postgres-host>
      - name: POSTGRES_PORT
        value: "5432"
      - name: POSTGRES_DB
        value: <postgres-database>
      - name: POSTGRES_USER
        value: <postgres-username>
      - name: POSTGRES_PASSWORD
        valueFrom:
          secretKeyRef:
            name: <postgres-secret-name>
            key: <postgres-password-key>

      # Inline embeddings (sentence-transformers)
      - name: ENABLE_SENTENCE_TRANSFORMERS
        value: "true"
      - name: EMBEDDING_PROVIDER
        value: "sentence-transformers"

      # Remote LLM configuration
      - name: INFERENCE_MODEL
        valueFrom:
          secretKeyRef:
            name: llama-stack-secret
```

```

    key: INFERENCE_MODEL
- name: VLLM_MAX_TOKENS
  value: "4096"
- name: VLLM_URL
  valueFrom:
    secretKeyRef:
      name: llama-stack-secret
      key: VLLM_URL
- name: VLLM_TLS_VERIFY
  valueFrom:
    secretKeyRef:
      name: llama-stack-secret
      key: VLLM_TLS_VERIFY
- name: VLLM_API_TOKEN
  valueFrom:
    secretKeyRef:
      name: llama-stack-secret
      key: VLLM_API_TOKEN

# Remote Milvus configuration from secret
- name: MILVUS_ENDPOINT
  valueFrom:
    secretKeyRef:
      name: milvus-secret
      key: MILVUS_ENDPOINT
- name: MILVUS_TOKEN
  valueFrom:
    secretKeyRef:
      name: milvus-secret
      key: MILVUS_TOKEN
- name: MILVUS_CONSISTENCY_LEVEL
  valueFrom:
    secretKeyRef:
      name: milvus-secret
      key: MILVUS_CONSISTENCY_LEVEL
name: llama-stack
port: 8321
distribution:
  name: rh-dev

```



NOTE

The **rh-dev** value is an internal image reference. When you create the **LlamaStackDistribution** custom resource, the OpenShift AI Operator automatically resolves **rh-dev** to the container image in the appropriate registry. This internal image reference allows the underlying image to update without requiring changes to your custom resource.

4.1.6.3. Example C: LlamaStackDistribution with Inline FAISS

Use this example to enable the inline FAISS vector store. This example uses inline embeddings served with the sentence-transformers library.



NOTE

The **ENABLE_SENTENCE_TRANSFORMERS** and **EMBEDDING_PROVIDER** environment variables shown in the YAML below are required to enable the **inline::sentence-transformers** provider in OpenShift AI 3.4. Omitting them disables inline embeddings.

1. In the OpenShift web console, select **Administrator** → **Quick Create** () → **Import YAML**, and create a CR similar to the following:

```
apiVersion: llamastack.io/v1alpha1
kind: LlamaStackDistribution
metadata:
  name: lsd-llama-faiss-inline
spec:
  replicas: 1
  server:
    containerSpec:
      resources:
        requests:
          cpu: "250m"
          memory: "500Mi"
        limits:
          cpu: "8"
          memory: "12Gi"
    env:
      # PostgreSQL metadata store (required in {productname-short} 3.2)
      - name: POSTGRES_HOST
        value: <postgres-host>
      - name: POSTGRES_PORT
        value: "5432"
      - name: POSTGRES_DB
        value: <postgres-database>
      - name: POSTGRES_USER
        value: <postgres-username>
      - name: POSTGRES_PASSWORD
        valueFrom:
          secretKeyRef:
            name: <postgres-secret-name>
            key: <postgres-password-key>

      # Inline embeddings (sentence-transformers)
      - name: ENABLE_SENTENCE_TRANSFORMERS
        value: "true"
      - name: EMBEDDING_PROVIDER
        value: "sentence-transformers"

      # Remote LLM configuration
      - name: INFERENCE_MODEL
        valueFrom:
          secretKeyRef:
            name: llama-stack-secret
            key: INFERENCE_MODEL
      - name: VLLM_URL
        valueFrom:
```

```

    secretKeyRef:
      name: llama-stack-secret
      key: VLLM_URL
- name: VLLM_TLS_VERIFY
  valueFrom:
    secretKeyRef:
      name: llama-stack-secret
      key: VLLM_TLS_VERIFY
- name: VLLM_API_TOKEN
  valueFrom:
    secretKeyRef:
      name: llama-stack-secret
      key: VLLM_API_TOKEN

# Enable inline FAISS
- name: ENABLE_FAISS
  value: "true"

- name: FMS_ORCHESTRATOR_URL
  value: "http://localhost"
name: llama-stack
port: 8321
distribution:
  name: rh-dev

```



NOTE

The **rh-dev** value is an internal image reference. When you create the **LlamaStackDistribution** custom resource, the OpenShift AI Operator automatically resolves **rh-dev** to the container image in the appropriate registry. This internal image reference allows the underlying image to update without requiring changes to your custom resource.

4.1.6.4. Example D: LlamaStackDistribution with Remote PostgreSQL with pgvector

Use this example when you want to use a PostgreSQL database with the pgvector extension as the vector store backend. This configuration enables the pgvector provider and reads connection values from a secret. This example uses remote embeddings.

1. Create the pgvector connection secret:

```

export PGVECTOR_HOST="<pgvector-hostname>"
export PGVECTOR_PORT="5432"
export PGVECTOR_DB="<pgvector-database>"
export PGVECTOR_USER="<pgvector-username>"
export PGVECTOR_PASSWORD="<pgvector-password>"

oc create secret generic pgvector-connection -n <project-name> \
  --from-literal=PGVECTOR_HOST="$PGVECTOR_HOST" \
  --from-literal=PGVECTOR_PORT="$PGVECTOR_PORT" \
  --from-literal=PGVECTOR_DB="$PGVECTOR_DB" \
  --from-literal=PGVECTOR_USER="$PGVECTOR_USER" \
  --from-literal=PGVECTOR_PASSWORD="$PGVECTOR_PASSWORD"

```

2. In the OpenShift web console, select **Administrator** → **Quick Create** () → **Import YAML**, and create a custom resource similar to the following:

```

apiVersion: llamastack.io/v1alpha1
kind: LlamaStackDistribution
metadata:
  name: lsdl-llama-pgvector-remote
spec:
  replicas: 1
  server:
    containerSpec:
      resources:
        requests:
          cpu: "250m"
          memory: "500Mi"
        limits:
          cpu: 4
          memory: "12Gi"
      env:
        # PostgreSQL metadata store (required in {productname-short} 3.2)
        - name: POSTGRES_HOST
          value: <postgres-host>
        - name: POSTGRES_PORT
          value: "5432"
        - name: POSTGRES_DB
          value: <postgres-database>
        - name: POSTGRES_USER
          value: <postgres-username>
        - name: POSTGRES_PASSWORD
          valueFrom:
            secretKeyRef:
              name: <postgres-secret-name>
              key: <postgres-password-key>

        # Remote LLM configuration
        - name: INFERENCE_MODEL
          valueFrom:
            secretKeyRef:
              name: llama-stack-secret
              key: INFERENCE_MODEL
        - name: VLLM_URL
          valueFrom:
            secretKeyRef:
              name: llama-stack-secret
              key: VLLM_URL
        - name: VLLM_TLS_VERIFY
          valueFrom:
            secretKeyRef:
              name: llama-stack-secret
              key: VLLM_TLS_VERIFY
        - name: VLLM_API_TOKEN
          valueFrom:
            secretKeyRef:
              name: llama-stack-secret
              key: VLLM_API_TOKEN

```

```

- name: VLLM_MAX_TOKENS
  valueFrom:
    secretKeyRef:
      name: llama-stack-secret
      key: VLLM_MAX_TOKENS

# Remote embedding configuration
- name: EMBEDDING_MODEL
  valueFrom:
    secretKeyRef:
      name: llama-stack-secret
      key: EMBEDDING_MODEL
- name: EMBEDDING_PROVIDER_MODEL_ID
  valueFrom:
    secretKeyRef:
      name: llama-stack-secret
      key: EMBEDDING_PROVIDER_MODEL_ID
- name: VLLM_EMBEDDING_URL
  valueFrom:
    secretKeyRef:
      name: llama-stack-secret
      key: VLLM_EMBEDDING_URL
- name: VLLM_EMBEDDING_TLS_VERIFY
  valueFrom:
    secretKeyRef:
      name: llama-stack-secret
      key: VLLM_EMBEDDING_TLS_VERIFY
- name: VLLM_EMBEDDING_API_TOKEN
  valueFrom:
    secretKeyRef:
      name: llama-stack-secret
      key: VLLM_EMBEDDING_API_TOKEN
- name: VLLM_EMBEDDING_MAX_TOKENS
  valueFrom:
    secretKeyRef:
      name: llama-stack-secret
      key: VLLM_EMBEDDING_MAX_TOKENS

# Enable and configure pgvector provider
- name: ENABLE_PGVECTOR
  value: "true"
- name: PGVECTOR_HOST
  valueFrom:
    secretKeyRef:
      name: pgvector-connection
      key: PGVECTOR_HOST
- name: PGVECTOR_PORT
  valueFrom:
    secretKeyRef:
      name: pgvector-connection
      key: PGVECTOR_PORT
- name: PGVECTOR_DB
  valueFrom:
    secretKeyRef:
      name: pgvector-connection
      key: PGVECTOR_DB

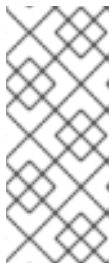
```

```

- name: PGVECTOR_USER
  valueFrom:
    secretKeyRef:
      name: pgvector-connection
      key: PGVECTOR_USER
- name: PGVECTOR_PASSWORD
  valueFrom:
    secretKeyRef:
      name: pgvector-connection
      key: PGVECTOR_PASSWORD

- name: FMS_ORCHESTRATOR_URL
  value: "http://localhost"
name: llama-stack
port: 8321
distribution:
  name: rh-dev

```



NOTE

The **rh-dev** value is an internal image reference. When you create the **LlamaStackDistribution** custom resource, the OpenShift AI Operator automatically resolves **rh-dev** to the container image in the appropriate registry. This internal image reference allows the underlying image to update without requiring changes to your custom resource.

3. Click **Create**.

Verification

- In the left-hand navigation, click **Workloads → Pods** and verify that the Llama Stack pod is running in the correct namespace.
- To verify that the Llama Stack server is running, click the pod name and select the **Logs** tab. Look for output similar to the following:

```

INFO 2025-05-15 11:23:52,750 __main__:498 server: Listening on [':', '0.0.0.0']:8321
INFO: Started server process [1]
INFO: Waiting for application startup.
INFO 2025-05-15 11:23:52,765 __main__:151 server: Starting up
INFO: Application startup complete.
INFO: Uvicorn running on http://[':', '0.0.0.0']:8321 (Press CTRL+C to quit)

```

TIP

If you switch between vector store configurations, delete the existing pod to ensure the new environment variables and backing store are picked up cleanly.

4.1.7. Ingesting content into a Llama model

You can quickly customize and prototype retrievable content by uploading a document and adding it to a vector store from inside a Jupyter notebook. This approach avoids building a separate ingestion pipeline. By using the Llama Stack SDK, you can ingest documents into a vector store and enable retrieval-augmented generation (RAG) workflows.

Prerequisites

- You have installed OpenShift 4.19 or newer.
- You have deployed a Llama 3.2 model with a vLLM model server.
- You have created a **LlamaStackDistribution** instance.
- You have configured a PostgreSQL database for Llama Stack metadata storage.
- You have configured an embedding model:
 - **Recommended:** You have configured a remote embedding model by using environment variables in the **LlamaStackDistribution**.
 - **Optional:** You have enabled inline embeddings with the sentence-transformers library for development or testing.
- You have created a workbench within a project.
- You have opened a Jupyter notebook and it is running in your workbench environment.
- You have installed **llama_stack_client** version 0.3.1 or later in your workbench environment.
- You have installed **requests** in your workbench environment. This is required for downloading example documents.
- If you use a remote vector store or remote embedding model, your environment has network access to those services through OpenShift.

Procedure

1. In a new notebook cell, install the client:

```
%pip install llama_stack_client
```

2. Install the **requests** library if it is not already available:

```
%pip install requests
```

3. Import **LlamaStackClient** and create a client instance:

```
from llama_stack_client import LlamaStackClient

# Use the Llama Stack service or route URL that is reachable from the workbench.
# Do not append /v1 when using llama_stack_client.
client = LlamaStackClient(base_url="<llama-stack-base-url>")
```

4. List the available models:

```
models = client.models.list()
```

5. Verify that the list includes:

- At least one LLM model.

- At least one embedding model (remote or inline).

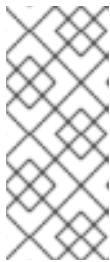
```
[Model(identifier='llama-32-3b-instruct', model_type='llm', provider_id='vllm-inference'),
 Model(identifier='nomic-embed-text-v1-5', model_type='embedding', metadata=
 {'embedding_dimension': 768})]
```

6. Select one LLM and one embedding model:

```
model_id = next(m.identifier for m in models if m.model_type == "llm")

embedding_model = next(m for m in models if m.model_type == "embedding")
embedding_model_id = embedding_model.identifier
embedding_dimension = int(embedding_model.metadata["embedding_dimension"])
```

7. (Optional) Create a vector store. Skip this step if you already have one.



NOTE

Provider IDs can differ between interfaces. In the Python SDK, you typically use the provider name directly (for example, **provider_id: "pgvector"**). In some CLI tools and examples, remote providers might use a prefixed identifier (for example, **--vector-db-provider-id remote-pgvector**). Use the provider ID format that matches the interface you are using.

Example 4.1. Option 1: Inline Milvus (embedded)

```
vector_store = client.vector_stores.create(
    name="my_inline_milvus",
    extra_body={
        "embedding_model": embedding_model_id,
        "embedding_dimension": embedding_dimension,
        "provider_id": "milvus",
    },
)
vector_store_id = vector_store.id
```



NOTE

Inline Milvus is suitable for development and small datasets. In OpenShift AI 3.2 and later, metadata persistence uses PostgreSQL by default.

Example 4.2. Option 2: Remote Milvus (recommended for production)

```
vector_store = client.vector_stores.create(
    name="my_remote_milvus",
    extra_body={
        "embedding_model": embedding_model_id,
        "embedding_dimension": embedding_dimension,
        "provider_id": "milvus-remote",
    },
)
```

```

    },
  )
  vector_store_id = vector_store.id

```

**NOTE**

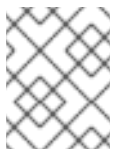
Ensure your **LlamaStackDistribution** is configured with **MILVUS_ENDPOINT** and **MILVUS_TOKEN**.

Example 4.3. Option 3: Inline FAISS

```

vector_store = client.vector_stores.create(
    name="my_inline_faiss",
    extra_body={
        "embedding_model": embedding_model_id,
        "embedding_dimension": embedding_dimension,
        "provider_id": "faiss",
    },
)
vector_store_id = vector_store.id

```

**NOTE**

Inline FAISS is an in-process vector store intended for development and testing. In OpenShift AI 3.2 and later, FAISS uses PostgreSQL as the default metadata store.

Example 4.4. Option 4: Remote PostgreSQL with pgvector

```

vector_store = client.vector_stores.create(
    name="my_pgvector_store",
    extra_body={
        "embedding_model": embedding_model_id,
        "embedding_dimension": embedding_dimension,
        "provider_id": "pgvector",
    },
)
vector_store_id = vector_store.id

```

**NOTE**

Ensure that the pgvector provider is enabled in your **LlamaStackDistribution** and that the PostgreSQL instance has the pgvector extension installed.

1. If you already have a vector store, set its identifier:

```
# vector_store_id = "<existing-vector-store-id>"
```

2. Download a PDF, upload it to Llama Stack, and add it to your vector store:

```
import requests

pdf_url = "https://www.federalreserve.gov/aboutthefed/files/quarterly-report-20250822.pdf"
filename = "quarterly-report-20250822.pdf"

response = requests.get(pdf_url)
response.raise_for_status()

with open(filename, "wb") as f:
    f.write(response.content)

with open(filename, "rb") as f:
    file_info = client.files.create(
        file=filename, f,
        purpose="assistants",
    )

vector_store_file = client.vector_stores.files.create(
    vector_store_id=vector_store_id,
    file_id=file_info.id,
    chunking_strategy={
        "type": "static",
        "static": {
            "max_chunk_size_tokens": 800,
            "chunk_overlap_tokens": 400,
        },
    },
)

print(vector_store_file)
```

Verification

- The call to **client.vector_stores.files.create()** succeeds and returns metadata for the ingested file.
- The vector store contains indexed chunks associated with the uploaded document.
- Subsequent RAG queries can retrieve content from the vector store.

4.1.8. Querying ingested content in a Llama model

You can use the Llama Stack SDK in your Jupyter notebook to query ingested content by running retrieval-augmented generation (RAG) queries on content stored in your vector store. You can perform one-off lookups without setting up a separate retrieval service.

Prerequisites

- You have installed OpenShift 4.19 or newer.
- You have enabled GPU support in OpenShift AI. This includes installing the Node Feature Discovery Operator and NVIDIA GPU Operator. For more information, see [Installing the Node Feature Discovery Operator](#) and [Enabling NVIDIA GPUs](#).

- If you are using GPU acceleration, you have at least one NVIDIA GPU available.
- You have activated the Llama Stack Operator in OpenShift AI.
- You have deployed an inference model, for example, the **llama-3.2-3b-instruct** model.
- You have created a **LlamaStackDistribution** instance with:
 - PostgreSQL configured as the metadata store.
 - An embedding model configured, preferably as a remote embedding provider.
- You have created a workbench within a project and opened a running Jupyter notebook.
- You have installed **llama_stack_client** version 0.3.1 or later in your workbench environment.
- You have already ingested content into a vector store.



NOTE

This procedure requires that content has already been ingested into a vector store. If no content is available, RAG queries return empty or non-contextual responses.

Procedure

1. In a new notebook cell, install the client:

```
%pip install -q llama_stack_client
```

2. Import **LlamaStackClient**:

```
from llama_stack_client import LlamaStackClient
```

3. Create a client instance:

```
# Use the Llama Stack service or route URL that is reachable from the workbench.
# Do not append /v1 when using llama_stack_client.
client = LlamaStackClient(base_url="<llama-stack-base-url>")
```

4. List available models:

```
models = client.models.list()
```

5. Select an LLM. If you plan to register a new vector store, also capture an embedding model:

```
model_id = next(m.identifier for m in models if m.model_type == "llm")

embedding = next((m for m in models if m.model_type == "embedding"), None)
if embedding:
    embedding_model_id = embedding.identifier
    embedding_dimension = int(embedding.metadata.get("embedding_dimension", 768))
```

6. If you do not already have a vector store ID, register a vector store (choose one):

Example 4.5. Option 1: Inline Milvus (embedded)

```
vector_store = client.vector_stores.create(
    name="my_inline_milvus",
    extra_body={
        "embedding_model": embedding_model_id,
        "embedding_dimension": embedding_dimension,
        "provider_id": "milvus",
    },
)
vector_store_id = vector_store.id
```

**NOTE**

Inline Milvus is suitable for development and small datasets. In OpenShift AI 3.2 and later, metadata persistence uses PostgreSQL by default.

Example 4.6. Option 2: Remote Milvus (recommended for production)

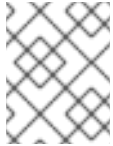
```
vector_store = client.vector_stores.create(
    name="my_remote_milvus",
    extra_body={
        "embedding_model": embedding_model_id,
        "embedding_dimension": embedding_dimension,
        "provider_id": "milvus-remote",
    },
)
vector_store_id = vector_store.id
```

**NOTE**

Ensure your **LlamaStackDistribution** sets **MILVUS_ENDPOINT** (gRPC port 19530) and **MILVUS_TOKEN**.

Example 4.7. Option 3: Inline FAISS

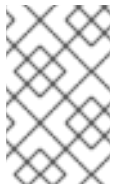
```
vector_store = client.vector_stores.create(
    name="my_inline_faiss",
    extra_body={
        "embedding_model": embedding_model_id,
        "embedding_dimension": embedding_dimension,
        "provider_id": "faiss",
    },
)
vector_store_id = vector_store.id
```

**NOTE**

Inline FAISS is an in-process vector store intended for development and testing. In OpenShift AI 3.2 and later, FAISS uses PostgreSQL as the default metadata store.

Example 4.8. Option 4: Remote PostgreSQL with pgvector

```
vector_store = client.vector_stores.create(
    name="my_pgvector_store",
    extra_body={
        "embedding_model": embedding_model_id,
        "embedding_dimension": embedding_dimension,
        "provider_id": "pgvector",
    },
)
vector_store_id = vector_store.id
```

**NOTE**

Ensure the pgvector provider is enabled in your **LlamaStackDistribution** and that the PostgreSQL instance has the pgvector extension installed. This option is suitable for production-grade RAG workloads that require durability and concurrency.

1. If you already have a vector store, set its identifier:

```
# vector_store_id = "<existing-vector-store-id>"
```

2. Query without using a vector store:

```
system_instructions = """You are a precise and reliable AI assistant.
Use retrieved context when it is available.
If nothing relevant is found, say so clearly."""

query = "How do you do great work?"

response = client.responses.create(
    model=model_id,
    input=query,
    instructions=system_instructions,
)

print(response.output_text)
```

3. Query by using the Responses API with file search:

```
response = client.responses.create(
    model=model_id,
    input=query,
    instructions=system_instructions,
    tools=[
        {
```

```

        "type": "file_search",
        "vector_store_ids": [vector_store_id],
    },
],
)

print(response.output_text)

```



NOTE

When you include the **file_search** tool with **vector_store_ids**, Llama Stack retrieves relevant chunks from the specified vector store and provides them to the model as context for the response.

Verification

- The notebook returns a response without vector stores and a context-aware response when vector stores are enabled.
- No errors appear, confirming successful retrieval and model execution.

4.1.9. Preparing documents with Docling for Llama Stack retrieval

You can transform your source documents with a Docling-enabled pipeline and ingest the output into a Llama Stack vector store by using the Llama Stack SDK. This modular approach separates document preparation from ingestion while still enabling an end-to-end, retrieval-augmented generation (RAG) workflow.

The pipeline registers a vector store and downloads the source PDFs, then splits them for parallel processing and converts each batch to Markdown with Docling. It generates embeddings from the Markdown and stores them in the vector store, making the documents searchable through Llama Stack.

Prerequisites

- You have installed OpenShift 4.19 or newer.
- You have enabled GPU support in OpenShift AI. This includes installing the Node Feature Discovery operator and NVIDIA GPU Operators. For more information, see [Installing the Node Feature Discovery operator](#) and [Enabling NVIDIA GPUs](#).
- You have logged in to the OpenShift web console.
- You have a project and access to pipelines in the OpenShift AI dashboard.
- You have created and configured a pipeline server within the project that contains your workbench.
- You have activated the Llama Stack Operator in OpenShift AI.
- You have deployed an inference model, for example, the **llama-3.2-3b-instruct** model.
- You have configured a Llama Stack deployment by creating a **LlamaStackDistribution** instance to enable RAG functionality.
- You have created a workbench within a project.

- You have opened a Jupyter notebook and it is running in your workbench environment.
- You have installed the **llama_stack_client** version 0.3.1 or later in your workbench environment.
- You have installed local object storage buckets and created connections, as described in [Adding a connection to your project](#).
- You have compiled to YAML a pipeline that includes a Docling transform, either one of the RAG demo samples or your own custom pipeline.
- Your project quota allows between 500 millicores (0.5 CPU) and 4 CPU cores for the pipeline run.
- Your project quota allows from 2 GiB up to 6 GiB of RAM for the pipeline run.
- If you are using GPU acceleration, you have at least one NVIDIA GPU available.

Procedure

1. In a new notebook cell, install the client:

```
%pip install -q llama_stack_client
```

2. In a new notebook cell, import **LlamaStackClient**:

```
from llama_stack_client import LlamaStackClient
```

3. In a new notebook cell, assign your deployment endpoint to the **base_url** parameter to create a **LlamaStackClient** instance:

```
client = LlamaStackClient(base_url="http://<llama-stack-service>:8321")
```



NOTE

LlamaStackClient requires the service root without the **/v1** path suffix. For example, use **http://llama-stack-service:8321**.

The **/v1** suffix is required only when you use OpenAI-compatible SDKs or send raw HTTP requests to the OpenAI-compatible API surface.

4. List the available models:

```
models = client.models.list()
```

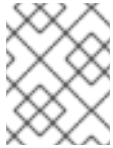
5. Select the first LLM and the first embedding model:

```
model_id = next(m.identifier for m in models if m.model_type == "llm")
embedding_model = next(m for m in models if m.model_type == "embedding")
embedding_model_id = embedding_model.identifier
embedding_dimension = int(embedding_model.metadata.get("embedding_dimension", 768))
```

6. Register a vector store (choose one option). Skip this step if your pipeline registers the store automatically.

Example 4.9. Option 1: Inline Milvus Lite (embedded)

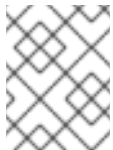
```
vector_store_name = "my_inline_db"
vector_store = client.vector_stores.create(
    name=vector_store_name,
    extra_body={
        "embedding_model": embedding_model_id,
        "embedding_dimension": embedding_dimension,
        "provider_id": "milvus", # inline Milvus Lite
    },
)
vector_store_id = vector_store.id
print(f"Registered inline Milvus Lite DB: {vector_store_id}")
```

**NOTE**

Inline Milvus Lite is best for development. Data durability and scale are limited compared to remote Milvus.

Example 4.10. Option 2: Remote Milvus (recommended for production)

```
vector_store_name = "my_remote_db"
vector_store = client.vector_stores.create(
    name=vector_store_name,
    extra_body={
        "embedding_model": embedding_model_id,
        "embedding_dimension": embedding_dimension,
        "provider_id": "milvus-remote", # remote Milvus provider
    },
)
vector_store_id = vector_store.id
print(f"Registered remote Milvus DB: {vector_store_id}")
```

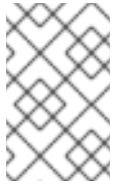
**NOTE**

Ensure your **LlamaStackDistribution** includes **MILVUS_ENDPOINT** and **MILVUS_TOKEN** (gRPC :19530).

Example 4.11. Option 3: Inline FAISS

```
vector_store_name = "my_faiss_db"
vector_store = client.vector_stores.create(
    name=vector_store_name,
    extra_body={
        "embedding_model": embedding_model_id,
        "embedding_dimension": embedding_dimension,
        "provider_id": "faiss", # inline FAISS provider
    },
)
```

```
)
vector_store_id = vector_store.id
print(f"Registered inline FAISS DB: {vector_store_id}")
```



NOTE

Inline FAISS (available in OpenShift AI 3.0 and later) is a lightweight, in-process vector store. It is best for local experimentation, disconnected environments, or single-node RAG deployments.



IMPORTANT

If you are using the sample Docling pipeline from the RAG demo repository, the pipeline registers the vector store automatically and you can skip the previous step. If you are using your own pipeline, you must register the vector store yourself.

1. In the OpenShift web console, import the YAML file containing your Docling pipeline into your project, as described in [Importing a pipeline](#).
2. Create a pipeline run to execute your Docling pipeline, as described in [Executing a pipeline run](#). The pipeline run inserts your PDF documents into the vector store. If you run the Docling pipeline from the [RAG demo samples repository](#), you can optionally customize the following parameters before starting the pipeline run:
 - **base_url**: The base URL to fetch PDF files from.
 - **pdf_filenames**: A comma-separated list of PDF filenames to download and convert.
 - **num_workers**: The number of parallel workers.
 - **vector_store_id**: The vector store identifier.
 - **service_url**: The Milvus service URL (only for remote Milvus).
 - **embed_model_id**: The embedding model to use.
 - **max_tokens**: The maximum tokens for each chunk.
 - **use_gpu**: Enable or disable GPU acceleration.

Verification

1. In your Jupyter notebook, query the LLM with a question that relates to the ingested content:

```
system_instructions = """You are a precise and reliable AI assistant.
Use retrieved context when it is available.
If nothing relevant is found in the available files, say so clearly."""

prompt = "What can you tell me about the birth of word processing?"

# Query using the Responses API with file search
response = client.responses.create(
    model=model_id,
```

```

    input=prompt,
    instructions=system_instructions,
    tools=[
        {
            "type": "file_search",
            "vector_store_ids": [vector_store_id],
        }
    ],
)

print("Answer (with vector stores):")
print(response.output_text)

```

2. Query chunks from the vector store:

```

query_result = client.vector_io.query(
    vector_store_id=vector_store_id,
    query="word processing",
)
print(query_result)

```

- The pipeline run completes successfully in your project.
- Document embeddings are stored in the vector store and are available for retrieval.
- No errors or warnings appear in the pipeline logs or your notebook output.

4.1.10. About Llama stack search types

Llama Stack supports keyword, vector, and hybrid search modes for retrieving context in retrieval-augmented generation (RAG) workloads. Each mode offers different tradeoffs in precision, recall, semantic depth, and computational cost.

4.1.10.1. Supported search modes

4.1.10.1.1. Keyword search

Keyword search applies lexical matching techniques, such as TF-IDF or BM25, to locate documents that contain exact or near-exact query terms. This approach is effective when precise term-matching is required, such as searching for identifiers, names, or regulatory terms.

Keyword search example

```

query_result = client.vector_io.query(
    vector_store_id=vector_store_id,
    query="FRBNY",
    params={
        "mode": "keyword",
        "max_chunks": 3,
        "score_threshold": 0.7,
    },
)

print(query_result)

```

For more information about keyword-based retrieval, see [The Probabilistic Relevance Framework: BM25 and Beyond](#).

4.1.10.1.2. Vector search

Vector search encodes documents and queries as dense numerical vectors, known as embeddings, and measures similarity using metrics such as cosine similarity or inner product. This approach captures semantic meaning and supports contextual matching beyond exact word overlap.

Vector search example

```
query_result = client.vector_io.query(
    vector_store_id=vector_store_id,
    query="FRBNY",
    params={
        "mode": "vector",
        "max_chunks": 3,
        "score_threshold": 0.7,
    },
)

print(query_result)
```

For more information, see [Billion-scale similarity search with GPUs](#).

4.1.10.1.3. Hybrid search

Hybrid search combines keyword and vector-based retrieval techniques, typically by blending lexical and semantic relevance scores. This approach aims to balance exact term matching with semantic similarity.

Hybrid search example

```
query_result = client.vector_io.query(
    vector_store_id=vector_store_id,
    query="FRBNY",
    params={
        "mode": "hybrid",
        "max_chunks": 3,
        "score_threshold": 0.7,
    },
)

print(query_result)
```

For more information, see [Sparse, Dense, and Hybrid Retrieval for Answer Ranking](#).



NOTE

Search mode availability depends on the selected vector store provider and its configured capabilities.

Not all providers support every search mode. For example, some providers might support vector search only, while keyword or hybrid search might be unavailable or return empty results. Always verify supported search modes for your chosen vector store provider.

Additional resources

- [Introduction to OpenShift AI](#)
- [Demystify RAG with OpenShift AI and Elasticsearch](#)
- [From Podman AI Lab to OpenShift AI](#)
- [Red Hat OpenShift AI learning](#)

4.2. EVALUATING RAG SYSTEMS WITH LLAMA STACK

You can use the evaluation providers that Llama Stack exposes to measure and improve the quality of your Retrieval-Augmented Generation (RAG) workloads in OpenShift AI. This section introduces RAG evaluation providers, describes how to use Ragas with Llama Stack, shows how to benchmark embedding models with BEIR, and helps you choose the right provider for your use case.

4.2.1. Understanding RAG evaluation providers

Llama Stack supports pluggable evaluation providers that measure the quality and performance of Retrieval-Augmented Generation (RAG) pipelines. Evaluation providers assess how accurately, faithfully, and relevantly the generated responses align with the retrieved context and the original user query. Each provider implements its own metrics and evaluation methodology. You can enable a specific provider through the configuration of the **LlamaStackDistribution** custom resource.

OpenShift AI supports the following evaluation providers:

- **Ragas:** A lightweight, Python-based framework that evaluates factuality, contextual grounding, and response relevance.
- **BEIR:** A benchmarking framework for retrieval performance across multiple datasets.
- **TrustyAI:** A Red Hat framework that evaluates explainability, fairness, and reliability of model outputs.

Evaluation providers operate independently of model serving and retrieval components. You can run evaluations asynchronously and aggregate results for quality tracking over time.

4.2.2. Using Ragas with Llama Stack

You can use the Ragas (Retrieval-Augmented Generation Assessment) evaluation provider with Llama Stack to measure the quality of your Retrieval-Augmented Generation (RAG) workflows in OpenShift AI. Ragas integrates with the Llama Stack evaluation API to compute metrics such as faithfulness, answer relevancy, and context precision for your RAG workloads.

Llama Stack exposes evaluation providers as part of its API surface. When you configure Ragas as a provider, the Llama Stack server sends RAG inputs and outputs to Ragas and records the resulting metrics for later analysis.

Ragas evaluation with Llama Stack in OpenShift AI supports the following deployment modes:

- **Inline provider** for development and small-scale experiments.
- **Remote provider** for production-scale evaluations that run as OpenShift AI AI pipelines.

You choose the mode that best fits your workflow:

- Use the inline provider when you want fast, low-overhead evaluation while you iterate on prompts, retrieval configuration, or model choices.
- Use the remote provider when you need to evaluate large datasets, integrate with CI/CD pipelines, or run repeated benchmarks at scale.

Evaluating RAG systems with RAGAS.

4.2.3. Benchmarking embedding models with BEIR datasets and Llama Stack

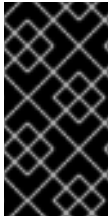
This procedure explains how to set up, run, and verify embedding model benchmarks by using the Llama Stack framework. Embedding models are neural networks that convert text or other data into dense numerical vectors called embeddings, which capture semantic meaning. In retrieval augmented generation systems, embeddings enable semantic search so that the system retrieves the documents most relevant to a query.

Selecting an embedding model depends on several factors, such as the content type, accuracy requirements, performance needs, and model license. The **beir_benchmarks.py** script compares the retrieval accuracy of embedding models by using standardized information retrieval benchmarks from the BEIR framework. The script is included in the [RAG](#) repository, which provides demonstrations, benchmarking scripts, and deployment guides for the RAG stack on OpenShift.

The examples use the **sentence-transformers** inference provider, which you can replace with another provider if required.

Prerequisites

- You have cloned the <https://github.com/opendatahub-io/rag> repository.
- You have changed into the **/rag/benchmarks/beir-benchmarks** directory.
- You have initialized and activated a virtual environment.
- You have defined and installed the relevant script package dependencies to a **requirements.txt** file.
- You have built the Llama Stack starter distribution to install all dependencies.
- You have verified that your vector database is accessible and configured in the **run.yaml** file, and that any required embedding models were preloaded or registered with Llama Stack.



IMPORTANT

Before you run the benchmark script, the Llama Stack server must be running and a vector database provider must be enabled and reachable. If you plan to compare embedding models beyond the default set, you must also register those embedding models with Llama Stack.



NOTE

The BEIR (Benchmarking Information Retrieval) datasets and evaluation scripts rely on the **beir** Python package. At this time, pre-compiled Python wheels for the beir library are not available for the IBM Power **ppc64le** architecture.

Procedure

1. Optional: Start the Llama Stack server and enable a vector database provider. If you have not already started Llama Stack with a vector database provider enabled, start the server by using a configuration similar to one of the following examples:

- Using inline Milvus:

```
MILVUS_URL=milvus uv run llama stack run run.yaml
```

- Using remote PostgreSQL with pgvector:

```
ENABLE_PGVECTOR=true PGVECTOR_DB=pgvector PGVECTOR_USER=<user>  
PGVECTOR_PASSWORD=<password> uv run llama stack run run.yaml
```

2. Optional: Register additional embedding models. The default supported embedding models are **granite-embedding-30m** and **granite-embedding-125m**, served by the **sentence-transformers** framework. If you want to benchmark additional embedding models, register them with Llama Stack before running the benchmark script.

For example, register an embedding model by using the Llama Stack client:

```
llama-stack-client models register all-MiniLM-L6-v2 \  
--provider-id sentence-transformers \  
--provider-model-id all-minilm:latest \  
--metadata {"embedding_dimension": 384} \  
--model-type embedding
```



NOTE

Any embedding models specified in the **--embedding-models** option must be registered before running the benchmark script.

3. Run the **beir_benchmarks.py** benchmarking script.

- Enter the following command to use the configuration from **run.yaml** and the default dataset **scifact** with inline Milvus:

```
MILVUS_URL=milvus uv run python beir_benchmarks.py
```

- Enter the following command to run the benchmark by using remote PostgreSQL with pgvector:

```
ENABLE_PGVECTOR=true PGVECTOR_DB=pgvector uv run python
beir_benchmarks.py \
  --vector-db-provider-id pgvector
```

- Alternatively, enter the following command to connect to a custom Llama Stack server:

```
LLAMA_STACK_URL="http://localhost:8321" MILVUS_URL=milvus uv run python
beir_benchmarks.py
```

4. Use environment variables and command line options to modify the benchmark run. For example, set the environment variable for the vector database provider before executing the script.

- Enter the following command to use a larger batch size for document ingestion:

```
MILVUS_URL=milvus uv run python beir_benchmarks.py --batch-size 300
```

- Enter the following command to benchmark multiple datasets, for example, **scifact** and **scidocs**:

```
MILVUS_URL=milvus uv run python beir_benchmarks.py \
  --dataset-names scifact scidocs
```

- Enter the following command to compare embedding models, for example, **granite-embedding-30m** and **all-MiniLM-L6-v2**:

```
MILVUS_URL=milvus uv run python beir_benchmarks.py \
  --embedding-models granite-embedding-30m all-MiniLM-L6-v2
```



NOTE

Ensure that **all-MiniLM-L6-v2** is registered with Llama Stack before running this command. See step 2 for registration instructions.

- Enter the following command to use a custom BEIR compatible dataset:

```
MILVUS_URL=milvus uv run python beir_benchmarks.py \
  --dataset-names my-dataset \
  --custom-datasets-urls https://example.com/my-beir-dataset.zip
```

- Enter the following command to change the vector database provider:

```
# Use remote PostgreSQL with pgvector
ENABLE_PGVECTOR=true PGVECTOR_DB=llama-stack PGVECTOR_USER=<user>
PGVECTOR_PASSWORD=<password> uv run python beir_benchmarks.py \
  --vector-db-provider-id pgvector
```

For information on command line options for benchmarking embedding models with BEIR datasets, see [Benchmarking embedding models with BEIR datasets and Llama Stack command line options](#).

Verification

To verify that the benchmark completed successfully and to review the results, perform the following steps:

1. Locate the **results** directory. All output files are saved to the following path:
<path-to>/rag/benchmarks/beir-benchmarks/results
2. Examine the output. Compare your results with the sample output structure. The report includes performance metrics such as **map@cut_k** and **ndcg@cut_k** for each dataset and embedding model pair. The script also calculates a statistical significance test called a p value.
Example output for scifact and map_cut_10:

```
scifact map_cut_10
granite-embedding-125m : 0.6879
granite-embedding-30m  : 0.6578
p_value                 : 0.0150
```

p_value < 0.05 indicates a statistically significant difference.
The granite-embedding-125m model performs better for this dataset and metric.

3. Interpret the results. A p value below **0.05** indicates that the performance difference between models is statistically significant. The model with the higher metric value performs better. Use these results to identify which embedding model performs best for your dataset.

4.2.3.1. BEIR benchmarking command-line options

The BEIR benchmarking script accepts the following command-line options:

- **--vector-db-provider-id**
 - **Description:** Specifies the vector database provider to use. You must enable the provider through the appropriate environment variable in the LlamaStackDistribution: milvus requires `ENABLE_INLINE_MILVUS`, faiss requires `ENABLE_FAISS`, and pgvector requires `ENABLE_PGVECTOR`.
 - **Type:** String.
 - **Default:** **milvus**.
 - **Example values:** **milvus**, **pgvector**, **faiss**.
 - **Example:**

```
--vector-db-provider-id pgvector
```
- **--dataset-names**
 - **Description:** Specifies which BEIR datasets to use for benchmarking. Use this option together with **--custom-datasets-urls** when testing custom datasets.
 - **Type:** List of strings.
 - **Default:** **["scifact"]**.

- **Example:**

```
--dataset-names scifact scidocs nq
```

- **--embedding-models**

- **Description:** Specifies the embedding models to compare. Models must be defined in the **run.yaml** file.
- **Type:** List of strings.
- **Default:** **["granite-embedding-30m", "granite-embedding-125m"]**.
- **Example:**

```
--embedding-models all-MiniLM-L6-v2 granite-embedding-125m
```

- **--batch-size**

- **Description:** Controls how many documents are processed per batch during ingestion. Larger batch sizes improve speed but use more memory.
- **Type:** Integer.
- **Default:** **150**.
- **Example:**

```
--batch-size 50
--batch-size 300
```

- **--custom-datasets-urls**

- **Description:** Specifies URLs for custom BEIR compatible datasets. Use this option with **--dataset-names**.
- **Type:** List of strings.
- **Default:** **[]**.
- **Example:**

```
--dataset-names my-custom-dataset \
--custom-datasets-urls https://example.com/my-dataset.zip
```

NOTE

Custom BEIR datasets must follow the required file structure and format:

```
dataset-name.zip/
├── qrels/
│   └── test.tsv
├── corpus.jsonl
└── queries.jsonl
```

For information on benchmarking embedding models with BEIR datasets, see [Benchmarking embedding models with BEIR datasets and Llama Stack](#).

Additional resources

- [Demystify RAG with OpenShift AI and Elasticsearch](#)
- [From Podman AI Lab to OpenShift AI](#)
- [Red Hat OpenShift AI learning](#)

4.3. USING POSTGRESQL IN LLAMA STACK

PostgreSQL is a dependency for Llama Stack deployments in OpenShift AI, where it serves as the mandatory metadata storage backend for supported vector storage configurations. Additionally, you can configure PostgreSQL as a remote vector database provider by enabling the pgvector extension.

In OpenShift AI, PostgreSQL serves the following roles in Llama Stack deployments:

- Required metadata storage for Llama Stack APIs and orchestration services.
- An optional remote vector database when the pgvector provider is enabled.

Depending on your deployment requirements, these roles can be fulfilled by the same PostgreSQL instance or separate instances. For example, you might use a single instance for development and testing environments, and separate instances for production deployments that require independent scaling or isolation.



IMPORTANT

The procedures provide basic configuration suitable for development and testing. Production deployments require additional planning, including the following considerations:

- High availability and replication
- Backup and disaster recovery
- Security hardening and encryption
- Performance tuning and monitoring

4.3.1. Understanding PostgreSQL in Llama Stack

4.3.1.1. Understanding Llama Stack metadata storage

In OpenShift AI, Llama Stack requires PostgreSQL as a metadata storage backend to persist state and configuration data across multiple components. Metadata storage provides durable persistence for vector stores, file management, agent state, conversation history, and other Llama Stack services.

PostgreSQL is required as a metadata storage backend for all OpenShift AI deployments.

4.3.1.1.1. Role of metadata storage in Llama Stack

Llama Stack components require persistent storage beyond in-memory data structures. Without metadata storage, component state would be lost on pod restarts or application failures.

Llama Stack uses metadata storage to persist:

- Vector store metadata, such as collection identifiers and document mappings.
- File metadata, including file locations, identifiers, and attributes.
- Agent state and conversation history.
- Dataset configurations and batch processing state.
- Model registry information and prompt templates.

This persistent storage allows Llama Stack to maintain operational state across pod restarts, rescheduling, and application updates.

4.3.1.1.2. PostgreSQL metadata storage backends

Llama Stack uses PostgreSQL to store multiple categories of metadata, including vector store metadata, file records, agent state, conversation history, and configuration data. These data types have different storage characteristics but are managed automatically within a single PostgreSQL instance.



IMPORTANT

PostgreSQL version 14 or later is required for all Llama Stack deployments, including development, testing, and production environments.

If validation errors occur, confirm that the deployed Llama Stack image version matches the configuration schema referenced by your **run.yaml**.

Llama Stack does not provision or manage the PostgreSQL instance used for metadata storage. You must deploy and manage the PostgreSQL database and supply its connection details when deploying Llama Stack.

4.3.2. Deploying and Configuring PostgreSQL

4.3.2.1. Deploying a PostgreSQL instance with pgvector

You can connect Llama Stack in OpenShift AI to an existing PostgreSQL instance that has the pgvector extension enabled. For development or evaluation, you can also deploy a PostgreSQL instance with the pgvector extension directly in your OpenShift project by creating Kubernetes resources through the OpenShift web console. This procedure focuses on deploying PostgreSQL with the pgvector extension for use as a remote vector store. It does not cover preparing a PostgreSQL database for use as Llama Stack metadata storage.

Prerequisites

- You have installed OpenShift 4.19 or newer.
- You have permissions to create resources in a project in your OpenShift cluster.
- You have PostgreSQL connection details available, including the database name, user name, and password.

- If you plan to deploy PostgreSQL in-cluster, you have a StorageClass that can provision persistent volumes.
- If you are using an existing PostgreSQL instance, the pgvector extension is installed and enabled on the target database.

Procedure

1. Log in to the OpenShift web console.
2. Select the project where you want to deploy the PostgreSQL instance.
3. Click the **Quick Create** () icon, and then click **Import YAML**.
4. Verify that the correct project is selected.
5. Copy the following YAML, replace the placeholder values, paste it into the YAML editor, and then click **Create**.



IMPORTANT

This example deploys a standalone PostgreSQL service with the pgvector extension enabled.

Llama Stack does not automatically use this database. To use this PostgreSQL instance as a vector store, you must explicitly configure the pgvector provider in a **LlamaStackDistribution**.

This example is intended for development or evaluation purposes. For production deployments, review and adapt the configuration to meet your organization's security, availability, backup, and lifecycle requirements.

Example PostgreSQL deployment with pgvector (development or evaluation)

```
apiVersion: v1
kind: Secret
metadata:
  name: <pgvector-postgresql-credentials-secret>
type: Opaque
stringData:
  POSTGRES_DB: "<database-name>"
  POSTGRES_USER: "<database-username>"
  POSTGRES_PASSWORD: "<database-password>"

---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: <pgvector-postgresql-pvc>
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
```

```

    storage: <storage-size>

---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: <pgvector-postgresql-deployment>
spec:
  replicas: 1
  selector:
    matchLabels:
      app: <pgvector-postgresql-app-label>
  template:
    metadata:
      labels:
        app: <pgvector-postgresql-app-label>
    spec:
      containers:
        - name: postgres
          image: pgvector/pgvector:pg16
          ports:
            - name: postgres
              containerPort: 5432
          env:
            - name: POSTGRES_DB
              valueFrom:
                secretKeyRef:
                  name: <pgvector-postgresql-credentials-secret>
                  key: POSTGRES_DB
            - name: POSTGRES_USER
              valueFrom:
                secretKeyRef:
                  name: <pgvector-postgresql-credentials-secret>
                  key: POSTGRES_USER
            - name: POSTGRES_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: <pgvector-postgresql-credentials-secret>
                  key: POSTGRES_PASSWORD
          volumeMounts:
            - name: pgdata
              mountPath: /var/lib/postgresql/data

      # Replace TCP socket probes with exec probes that validate SQL readiness.
      readinessProbe:
        exec:
          command:
            - /bin/sh
            - -c
            - pg_isready -h 127.0.0.1 -U "$POSTGRES_USER" -d "$POSTGRES_DB"
        initialDelaySeconds: 10
        periodSeconds: 10
        timeoutSeconds: 5
        failureThreshold: 6
      livenessProbe:
        exec:

```

```

    command:
      - /bin/sh
      - -c
      - pg_isready -h 127.0.0.1 -U "$POSTGRES_USER" -d "$POSTGRES_DB"
    initialDelaySeconds: 30
    periodSeconds: 20
    timeoutSeconds: 5
    failureThreshold: 6

    # Create the pgvector extension after PostgreSQL is actually accepting SQL.
    lifecycle:
      postStart:
        exec:
          command:
            - /bin/sh
            - -c
            - |
              set -e
              echo "Waiting for PostgreSQL to be ready before enabling pgvector..."
              until PGPASSWORD="$POSTGRES_PASSWORD" psql -h 127.0.0.1 -U
"$POSTGRES_USER" -d "$POSTGRES_DB" -c "SELECT 1" >/dev/null 2>&1; do
                sleep 2
              done
              PGPASSWORD="$POSTGRES_PASSWORD" psql -h 127.0.0.1 -U
"$POSTGRES_USER" -d "$POSTGRES_DB" -c "CREATE EXTENSION IF NOT EXISTS
vector;"

    volumes:
      - name: pgdata
        persistentVolumeClaim:
          claimName: <pgvector-postgresql-pvc>

---
apiVersion: v1
kind: Service
metadata:
  name: <pgvector-postgresql-service>
spec:
  selector:
    app: <pgvector-postgresql-app-label>
  ports:
    - name: postgres
      port: 5432
      targetPort: 5432
  type: ClusterIP

```

6. Click **Create**.

Verification

1. Navigate to **Networking → Services**.
2. Confirm that the PostgreSQL Service is listed and exposes port **5432**.
3. Navigate to **Workloads → Pods**.

4. Confirm that the PostgreSQL pod is running.



NOTE

This procedure verifies only that PostgreSQL with pgvector is deployed and reachable within the project. It does not verify integration with Llama Stack.

4.3.2.2. Configuring the pgvector remote provider in Llama Stack

To use PostgreSQL with the pgvector extension as a remote vector store, configure pgvector in your existing **LlamaStackDistribution** and provide PostgreSQL connection details as environment variables. Ensure that your **LlamaStackDistribution** already includes the PostgreSQL metadata storage configuration. This setup enables retrieval augmented generation (RAG) workflows in OpenShift AI by using PostgreSQL-based vector storage.

Prerequisites

- You have installed and enabled the Llama Stack Operator in OpenShift AI.
- You have a PostgreSQL database with the pgvector extension enabled. Llama Stack uses PostgreSQL for two purposes: metadata storage and the optional pgvector remote vector store. You can use a single PostgreSQL instance for both roles or deploy separate instances.
- You have the PostgreSQL connection details, including the host name, port number, database name, user name, and password.
- You have permissions to create Secrets and edit custom resources in your project.

Procedure

1. In the OpenShift web console, switch to the **Administrator** perspective.
2. Create a Secret that stores the PostgreSQL connection details.
 - a. Ensure that the correct project is selected.
 - b. Click **Workloads** → **Secrets**.
 - c. Click **Create** → **From YAML**.
 - d. Paste the following YAML, update the placeholder values, and then click **Create**.

Example Secret for pgvector connection details

```
apiVersion: v1
kind: Secret
metadata:
  name: pgvector-connection
type: Opaque
stringData:
  PGVECTOR_HOST: "<pgvector-hostname>"
  PGVECTOR_PORT: "<pgvector-port>"
  PGVECTOR_DB: "<database-name>"
  PGVECTOR_USER: "<database-username>"
  PGVECTOR_PASSWORD: "<database-password>"
```



IMPORTANT

The pgvector provider is not enabled automatically.

You must explicitly enable pgvector and supply its connection details through environment variables in your **LlamaStackDistribution**.

In OpenShift AI, the pgvector provider is enabled when the **ENABLE_PGVECTOR** environment variable is set.

3. Update your **LlamaStackDistribution** custom resource to enable pgvector and reference the Secret.
 - a. Click **Ecosystem** → **Installed Operators**.
 - b. Select the **Llama Stack Operator**.
 - c. Click the **LlamaStackDistribution** tab.
 - d. Select your **LlamaStackDistribution** resource.
 - e. Click **YAML**.
 - f. Update the resource to include the following fields, and then click **Save**.
Before you enable pgvector, deploy a Llama Stack server and configure the PostgreSQL metadata store.

For more information, see [Deploying a Llama Stack server](#).

Then update your existing **LlamaStackDistribution** to add the pgvector configuration shown in the following example. The example shows only the additional environment variables required to enable the pgvector provider.

Example LlamaStackDistribution configuration for pgvector

```
apiVersion: llamastack.io/v1alpha1
kind: LlamaStackDistribution
metadata:
  name: llamastack
spec:
  server:
    containerSpec:
      env:
        - name: ENABLE_PGVECTOR
          value: "true"
        - name: PGVECTOR_HOST
          valueFrom:
            secretKeyRef:
              name: pgvector-connection
              key: PGVECTOR_HOST
        - name: PGVECTOR_PORT
          valueFrom:
            secretKeyRef:
              name: pgvector-connection
              key: PGVECTOR_PORT
        - name: PGVECTOR_DB
```

```

valueFrom:
  secretKeyRef:
    name: pgvector-connection
    key: PGVECTOR_DB
- name: PGVECTOR_USER
  valueFrom:
    secretKeyRef:
      name: pgvector-connection
      key: PGVECTOR_USER
- name: PGVECTOR_PASSWORD
  valueFrom:
    secretKeyRef:
      name: pgvector-connection
      key: PGVECTOR_PASSWORD

```

Verification

1. Click **Workloads** → **Pods**.
2. Confirm that the Llama Stack pod restarts and reaches the **Running** state.
3. Open the pod logs and confirm that the server starts successfully and initializes the pgvector provider without errors.

4.4. USING QDRANT IN LLAMA STACK

Qdrant is a supported remote vector store provider for Llama Stack in OpenShift AI. You can deploy Qdrant in your OpenShift project or connect to an existing Qdrant instance, and configure Llama Stack to use Qdrant for retrieval-augmented generation (RAG) workloads.

To use Qdrant with Llama Stack, complete the following tasks:

- Review how Qdrant integrates with Llama Stack.
- Deploy a Qdrant instance or connect to an existing deployment.
- Configure your **LlamaStackDistribution** to use Qdrant as the vector store provider.
- Perform vector operations through the OpenAI-compatible Vector Stores API.

4.4.1. Overview of Qdrant vector databases

Qdrant is an open source vector database optimized for high-performance similarity search and advanced filtering. In OpenShift AI, Qdrant is supported as a remote vector store provider for Llama Stack and can be used in retrieval-augmented generation (RAG) workloads that require efficient vector indexing and durable storage.

When used with Llama Stack in OpenShift AI, Qdrant provides:

- High-performance similarity search using Hierarchical Navigable Small World (HNSW) indexing
- Filtering based on stored metadata during vector search
- Persistent storage of vector data
- Integration through the OpenAI-compatible Vector Stores API

In a RAG workflow:

- Embeddings are generated by the configured embedding provider.
- Qdrant stores embedding vectors and performs similarity search.
- Llama Stack manages ingestion, retrieval, and model inference through a unified API.

In OpenShift AI, you must deploy Qdrant as a remote service, either within your OpenShift project or as an externally managed deployment.



NOTE

Inline Qdrant is not supported. To use Qdrant with Llama Stack in OpenShift AI, deploy Qdrant as a remote service.

A typical remote deployment includes:

- A Qdrant service exposing HTTP (port 6333) and gRPC (port 6334) endpoints
- Persistent storage for vector data
- Optional API key authentication

For deployment and configuration instructions, see [Using Qdrant in Llama Stack](#).

4.4.2. Deploying a Qdrant vector database

You can connect Llama Stack in OpenShift AI to an existing Qdrant instance or deploy a Qdrant vector database in your OpenShift project. For development or evaluation purposes, you can deploy Qdrant by creating Kubernetes resources in the OpenShift web console.

Prerequisites

- You have installed OpenShift 4.19 or later.
- You have permission to create resources in a project.
- A StorageClass is available that can provision a PersistentVolume for the PersistentVolumeClaim used by this deployment.



NOTE

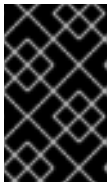
This example uses a single PersistentVolumeClaim. If your cluster uses dynamic provisioning, the StorageClass provisions the required PersistentVolume automatically.

- Optional: You have an API key for Qdrant authentication. If your Qdrant instance does not require authentication, remove the Secret and the **QDRANT__SERVICE__API_KEY** environment variable from the deployment example.

Procedure

1. Log in to the OpenShift web console.

2. From the **Project** list, select the project where you want to deploy Qdrant.
3. Click **Import YAML**.
4. Paste the following YAML:



IMPORTANT

This example deploys a standalone Qdrant service for development or evaluation. For production deployments, review and adapt the configuration to meet your organization's security, availability, backup, and lifecycle requirements.

```

apiVersion: v1
kind: Secret
metadata:
  name: <qdrant_credentials_secret>
type: Opaque
stringData:
  QDRANT_API_KEY: "<api_key>"

---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: <qdrant_pvc>
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: <storage_size>

---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: <qdrant_deployment>
spec:
  replicas: 1
  selector:
    matchLabels:
      app: <qdrant_app_label>
  template:
    metadata:
      labels:
        app: <qdrant_app_label>
    spec:
      containers:
        - name: qdrant
          image: qdrant/qdrant:v1.12.0
          ports:
            - name: http
              containerPort: 6333
            - name: grpc
              containerPort: 6334

```

```

env:
- name: QDRANT__SERVICE__API_KEY
  valueFrom:
    secretKeyRef:
      name: <qdrant_credentials_secret>
      key: QDRANT_API_KEY
volumeMounts:
- name: qdrant-storage
  mountPath: /qdrant/storage
- name: qdrant-storage
  mountPath: /qdrant/snapshots
  subPath: snapshots
readinessProbe:
  httpGet:
    path: /readyz
    port: 6333
  initialDelaySeconds: 5
  periodSeconds: 10
livenessProbe:
  httpGet:
    path: /healthz
    port: 6333
  initialDelaySeconds: 10
  periodSeconds: 20
volumes:
- name: qdrant-storage
  persistentVolumeClaim:
    claimName: <qdrant_pvc>

---
apiVersion: v1
kind: Service
metadata:
  name: <qdrant_service>
spec:
  selector:
    app: <qdrant_app_label>
  ports:
    - name: http
      port: 6333
      targetPort: 6333
    - name: grpc
      port: 6334
      targetPort: 6334
  type: ClusterIP

```



NOTE

If your Qdrant instance does not require authentication, remove the Secret and the **QDRANT__SERVICE__API_KEY** environment variable from the Deployment configuration.

5. Replace the placeholder values as follows:

- **<qdrant_credentials_secret>**: A name for the Secret that stores the Qdrant API key, for example `qdrant-credentials`

example **qdrant-credentials**.

- **<api_key>**: An API key for authenticating with Qdrant. If authentication is not required, remove the Secret and the **QDRANT__SERVICE__API_KEY** environment variable from the Deployment.
- **<qdrant_pvc>**: A name for the PersistentVolumeClaim, for example **qdrant-pvc**.
- **<storage_size>**: The storage capacity to request, for example **10Gi**.
- **<qdrant_deployment>**: A name for the Deployment, for example **qdrant**.
- **<qdrant_app_label>**: A label for the application, for example **qdrant**.
- **<qdrant_service>**: A name for the Service, for example **qdrant-service**.

6. Click **Create**.

Verification

- The Qdrant Service is present in the project and exposes ports **6333** (HTTP) and **6334** (gRPC). You can confirm this on the **Networking → Services** page in the OpenShift web console.
- The Qdrant pod reaches the **Running** state. You can confirm this on the **Workloads → Pods** page in the OpenShift web console.



NOTE

This verification confirms only that Qdrant is deployed and reachable within the project. To use this Qdrant instance with Llama Stack, configure the Qdrant provider in a **LlamaStackDistribution**.

4.4.3. Configuring the Qdrant remote provider in Llama Stack

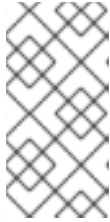
To use Qdrant as a remote vector store, configure your **LlamaStackDistribution** resource with the connection details for your Qdrant service. This configuration enables Llama Stack to store and retrieve embedding vectors using Qdrant in OpenShift AI.

Prerequisites

- You have installed and enabled the Llama Stack Operator in OpenShift AI.
- You have a running Qdrant instance that is accessible from your OpenShift cluster.
- You have the Qdrant connection details, including the service URL and, if required, an API key.
- You have permission to create Secrets and modify custom resources in your project.

Procedure

1. In the OpenShift web console, switch to the **Administrator** perspective.
2. Create a Secret that stores the Qdrant connection details used by Llama Stack. This Secret must contain the URL of the Qdrant service and, if required, the API key.

**NOTE**

If you deployed Qdrant by using the procedure in [Deploying a Qdrant vector database](#), create this Secret separately for the Llama Stack configuration. The Secret created during the Qdrant deployment does not contain the **QDRANT_URL** value required by the Llama Stack provider.

- a. From the **Project** list, select the project where the **LlamaStackDistribution** resource is deployed.
 - b. Click **Workloads → Secrets**.
 - c. Click **Create → From YAML**.
 - d. Paste the following YAML:


```
apiVersion: v1
kind: Secret
metadata:
  name: qdrant-connection
type: Opaque
stringData:
  QDRANT_URL: "<qdrant_url>"
  QDRANT_API_KEY: "<api_key>"
```
 - e. Replace the placeholder values as follows:
 - **<qdrant_url>**: The full URL to the Qdrant service, for example **http://qdrant-service:6333**. For in-cluster deployments, use the Service name and port. For external deployments, use the external URL.
 - **<api_key>**: The API key for authenticating with Qdrant. If authentication is not enabled for your Qdrant instance, remove the **QDRANT_API_KEY** entry from both the Secret and the **env** section in the **LlamaStackDistribution** configuration.
 - f. Click **Create**.
3. Update your **LlamaStackDistribution** custom resource to reference the Secret and supply the required environment variables.
 - a. Click **Operators → Installed Operators**.
 - b. Select the **Llama Stack Operator**.
 - c. Click the **LlamaStackDistribution** tab.
 - d. Select your **LlamaStackDistribution** resource.
 - e. Click **YAML**.
 - f. Update the resource to include the following fields.



NOTE

The environment variable names and configuration fields used by the Qdrant provider can vary depending on the Llama Stack version included with OpenShift AI. Before applying this configuration, verify that the variables and fields match the supported versions listed in [Supported Configurations for 3.x](#).

```
apiVersion: llamastack.io/v1alpha1
kind: LlamaStackDistribution
metadata:
  name: llamastack
spec:
  server:
    containerSpec:
      env:
        - name: ENABLE_QDRANT
          value: "true"
        - name: QDRANT_URL
          valueFrom:
            secretKeyRef:
              name: qdrant-connection
              key: QDRANT_URL
        - name: QDRANT_API_KEY
          valueFrom:
            secretKeyRef:
              name: qdrant-connection
              key: QDRANT_API_KEY
```

g. Click **Save**.

Verification

- The Llama Stack pod reaches the **Running** state. You can confirm this on the **Workloads → Pods** page in the OpenShift web console.
- The pod logs show that the Qdrant provider initializes successfully and does not report connection errors.
- Vector operations executed through the Llama Stack API complete successfully, confirming that Llama Stack can communicate with Qdrant.
For information about performing vector operations, see:

[Performing vector operations with Qdrant](#) .

4.4.4. Performing vector operations with Qdrant

After configuring Qdrant as the vector store provider in Llama Stack, you can perform vector operations by using the OpenAI-compatible Vector Stores API exposed by Llama Stack. These operations include creating vector stores, adding documents, performing similarity search, and deleting vector stores. You interact with the Llama Stack API rather than connecting directly to Qdrant. Llama Stack manages collection creation, embedding generation, and query execution on your behalf.

Prerequisites

- You have installed and enabled the Llama Stack Operator in OpenShift AI.
- You have configured Qdrant as the vector store provider in your **LlamaStackDistribution**.
- You have an embedding model available through a configured inference provider.
- You have network access to the Llama Stack API endpoint.
- You have installed the **jq** command-line utility.
For installation instructions, see [jq](#).
- You have the **curl** command-line tool installed.

Procedure

1. Determine how you will access the Llama Stack API.

You can access the API from within the cluster or from outside the cluster.

- **In-cluster access:** Run the **curl** commands from a pod in the same project, or from a workstation that has network access to the Llama Stack Service.
- **External access:** Expose the Llama Stack Service by creating a Route, and then use the Route URL from your local workstation.
For this procedure, set **LLAMA_STACK_URL** to the service or route root URL **without** the **/v1** suffix. The example commands append **/v1** as part of the endpoint path.

For more information about API compatibility and base URL requirements, see

[OpenAI compatibility for RAG APIs in Llama Stack](#) .

Example base URL for in-cluster access

```
LLAMA_STACK_URL="http://llamastack-service:8321"
```

Example base URL for external access through a Route

```
LLAMA_STACK_URL="https://llamastack-route.example.com"
```

2. Create a vector store and capture its ID.

```
CREATE_RESPONSE=$(curl -s -X POST "${LLAMA_STACK_URL}/v1/vector_stores" \
-H "Content-Type: application/json" \
-d '{
  "name": "my-rag-store",
  "embedding_model": "sentence-transformers/ibm-granite/granite-embedding-125m-english",
  "embedding_dimension": 768,
  "provider_id": "qdrant-remote"
}')
VECTOR_STORE_ID=$(echo "$CREATE_RESPONSE" | jq -r '.id')
echo "Vector store ID: ${VECTOR_STORE_ID}"
```

Ensure that the **VECTOR_STORE_ID** variable contains a valid value before continuing.

4.4.4.1. Add files to a vector store

Upload files to the vector store for ingestion. Llama Stack automatically splits the content into chunks, generates embeddings, and stores them in Qdrant.

Example using curl

```
FILE_RESPONSE=$(curl -s -X POST
"${LLAMA_STACK_URL}/v1/vector_stores/${VECTOR_STORE_ID}/files" \
-F "file=@/path/to/document.pdf" \
-F "purpose=assistants")

FILE_ID=$(echo "$FILE_RESPONSE" | jq -r '.id')
echo "File ID: ${FILE_ID}"
```

4.4.4.2. Query a vector store

Perform similarity search to retrieve relevant content from the vector store. The search query is converted into an embedding and compared with stored vectors in Qdrant.

Example using curl

```
curl -X POST "${LLAMA_STACK_URL}/v1/vector_stores/${VECTOR_STORE_ID}/search" \
-H "Content-Type: application/json" \
-d '{
  "query": "What is retrieval-augmented generation?",
  "max_results": 5
}'
```

4.4.4.3. Delete a vector store

Delete a vector store when it is no longer required. This removes the vector store and its associated data from Qdrant.

Example using curl

```
curl -X DELETE "${LLAMA_STACK_URL}/v1/vector_stores/${VECTOR_STORE_ID}"
```

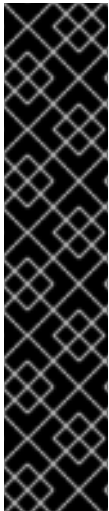
Verification

- Creating a vector store returns a valid vector store ID.
- File uploads complete successfully and are accepted by the API.
- Search queries return results from the ingested content.

4.5. CONFIGURING LLAMA STACK WITH OAUTH AUTHENTICATION

You can configure Llama Stack to use role-based access control (RBAC) for model access with OAuth authentication on OpenShift AI. The following example shows how to configure Llama Stack so that all authenticated users can access a vLLM model, while only specific users can access an OpenAI model. This example uses Keycloak to issue and validate tokens.

This procedure assumes that the Keycloak server is available at <https://my-keycloak-server.com>.



IMPORTANT

When you access Llama Stack APIs, the required base URL depends on the client that you use.

- For OpenAI-compatible clients or raw HTTP requests, include the **/v1** path suffix in the base URL.
For example, **<http://llama-stack-service:8321/v1>**
- For the **LlamaStackClient** SDK, do not include the **/v1** path suffix in the base URL.
For example, **<http://llama-stack-service:8321>**

If you use an incorrect base URL, requests fail.

Prerequisites

- You have installed OpenShift 4.19 or later.
- You have logged in to Red Hat OpenShift AI.
- You have cluster-admin privileges for your OpenShift cluster.
- You have a Keycloak instance configured with the following settings:
 - Realm: **llamastack-demo**
 - Client: **llamastack** with direct access grants enabled
 - Role: **inference_max**
 - A protocol mapper that adds realm roles to the access token under the **llamastack_roles** claim
 - Two test users:
 - **user1** with no assigned roles
 - **user2** assigned the **inference_max** role
- You have saved the Keycloak client secret for token requests.
- Your Keycloak server is reachable at **<https://my-keycloak-server.com>**.
- You have installed the OpenShift CLI (**oc**) as described in the documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS

Procedure

1. To configure Llama Stack to use role-based access control (RBAC) for model access, view and verify the OAuth provider token structure.

- a. Generate a Keycloak test token by running the following command:

```
$ curl -d client_id=llamastack -d client_secret=YOUR_CLIENT_SECRET -d
username=user1 -d password=user-password -d grant_type=password https://my-
keycloak-server.com/realms/llamastack-demo/protocol/openid-connect/token | jq -r
.access_token > test.token
```

- b. View the token claims by running the following command:

```
$ cat test.token | cut -d . -f 2 | base64 -d 2>/dev/null | jq .
```

Example token structure from Keycloak

```
{
  "iss": "https://my-keycloak-server.com/realms/llamastack-demo",
  "aud": "account",
  "sub": "761cdc99-80e5-4506-9b9e-26a67a8566f7",
  "preferred_username": "user1",
  "llamastack_roles": [
    "inference_max"
  ]
}
```

2. Update your existing **run.yaml** file to add the OAuth parameters.

Example OAuth parameters in the **run.yaml** file

```
server:
  port: 8321
  auth:
    provider_config:
      type: "oauth2_token"
    jwks:
      uri: "https://my-keycloak-server.com/realms/llamastack-demo/protocol/openid-
connect/certs" ❶
      key_recheck_period: 3600
    issuer: "https://my-keycloak-server.com/realms/llamastack-demo" ❷
    audience: "account"
    verify_tls: true
    claims_mapping:
      llamastack_roles: "roles" ❸
  access_policy:
    - permit: ❹
      actions: [read]
      resource: model::vllm-inference/llama-3-2-3b
      description: Allow all authenticated users to access the Llama 3.2 model
    - permit: ❺
      actions: [read]
      resource: model::openai/gpt-4o-mini
      when: user with inference_max in roles
      description: Allow only users with the inference_max role to access OpenAI models
```

❶ ❷ Specify your Keycloak host and realm in the URL.

- 3. Maps the **llamastack_roles** claim from the token to the **roles** field.
 - 4. Allows all authenticated users to access vLLM models.
 - 5. Restricts OpenAI models to users with the **inference_max** role.
3. Create a ConfigMap that uses the updated **run.yaml** configuration by running the following command:

```
$ oc create configmap llamastack-custom-config --from-file=run.yaml=run.yaml -n redhat-ods-operator
```

4. Create a **llamastack-distribution.yaml** file with the following content:

```
apiVersion: llamastack.io/v1alpha1
kind: LlamaStackDistribution
metadata:
  name: llamastack-distribution
  namespace: redhat-ods-operator
spec:
  replicas: 1
  server:
    distribution:
      name: rh-dev
    containerSpec:
      name: llama-stack
      port: 8321
      env:
        # vLLM provider configuration
        - name: VLLM_URL
          value: "https://your-vllm-service:8000/v1"
        - name: VLLM_API_TOKEN
          value: "your-vllm-token"
        - name: VLLM_TLS_VERIFY
          value: "false"
        # OpenAI provider configuration
        - name: OPENAI_API_KEY
          value: "your-openai-api-key"
        - name: OPENAI_BASE_URL
          value: "https://api.openai.com/v1"
      userConfig:
        configMapName: llamastack-custom-config
        configMapNamespace: redhat-ods-operator
```

5. Apply the distribution by running the following command:

```
$ oc apply -f llamastack-distribution.yaml
```

6. Wait for the distribution to be ready by running the following command:

```
$ oc wait --for=jsonpath='{.status.phase}'=Ready llamastackdistribution/llamastack-distribution -n redhat-ods-operator --timeout=300s
```

7. Generate OAuth tokens for each user account to authenticate API requests.

- a. To request a basic access token and save it to a **user1.token** file, run the following command:

```
$ curl -d client_id=llamastack \
  -d client_secret=YOUR_CLIENT_SECRET \
  -d username=user1 \
  -d password=user1-password \
  -d grant_type=password \
  https://my-keycloak-server.com/realms/llamastack-demo/protocol/openid-connect/token \
  | jq -r .access_token > user1.token
```

- b. To request a token for the privileged user and save it to a **user2.token** file, run the following command:

```
$ curl -d client_id=llamastack \
  -d client_secret=YOUR_CLIENT_SECRET \
  -d username=user2 \
  -d password=user2-password \
  -d grant_type=password \
  https://my-keycloak-server.com/realms/llamastack-demo/protocol/openid-connect/token \
  | jq -r .access_token > user2.token
```

- c. Verify the token claims by running the following command:

```
$ cat user2.token | cut -d . -f 2 | base64 -d 2>/dev/null | jq .
```

Verification

1. Set the Llama Stack service URL:

```
$ export LLAMASTACK_HOST="http://<llama-stack-host>:8321"
```

2. Verify basic access for **user1**, who has no privileged roles.
Load the token:

```
$ USER1_TOKEN=$(cat user1.token)
```

Confirm that **user1** can access the vLLM-served model:

```
$ curl -s -o /dev/null -w "%{http_code}\n" \
  -X POST "${LLAMASTACK_HOST}/v1/openai/chat/completions" \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer ${USER1_TOKEN}" \
  -d '{"model": "vllm-inference/llama-3-2-3b", "messages": \
  [{"role": "user", "content": "Hello!"}], "max_tokens": 50}'
```

Expected result: HTTP **200**.

Confirm that **user1** is denied access to the restricted OpenAI model:

```
$ curl -s -o /dev/null -w "%{http_code}\n" \
-X POST "${LLAMASTACK_HOST}/v1/openai/chat/completions" \
-H "Content-Type: application/json" \
-H "Authorization: Bearer ${USER1_TOKEN}" \
-d '{"model": "openai/gpt-4o-mini", "messages":
[{"role": "user", "content": "Hello!"}], "max_tokens": 50}'
```

Expected result: HTTP **403**.

3. Verify privileged access for **user2**, who is assigned the **inference_max** role.
Load the token:

```
$ USER2_TOKEN=$(cat user2.token)
```

Confirm that **user2** can access both models:

```
$ curl -s -o /dev/null -w "%{http_code}\n" \
-X POST "${LLAMASTACK_HOST}/v1/openai/chat/completions" \
-H "Content-Type: application/json" \
-H "Authorization: Bearer ${USER2_TOKEN}" \
-d '{"model": "vllm-inference/llama-3-2-3b", "messages":
[{"role": "user", "content": "Hello!"}], "max_tokens": 50}'
```

```
$ curl -s -o /dev/null -w "%{http_code}\n" \
-X POST "${LLAMASTACK_HOST}/v1/openai/chat/completions" \
-H "Content-Type: application/json" \
-H "Authorization: Bearer ${USER2_TOKEN}" \
-d '{"model": "openai/gpt-4o-mini", "messages":
[{"role": "user", "content": "Hello!"}], "max_tokens": 50}'
```

Expected result: HTTP **200** for both requests.

4. Verify that requests without a Bearer token are denied.

```
$ curl -s -o /dev/null -w "%{http_code}\n" \
-X POST "${LLAMASTACK_HOST}/v1/openai/chat/completions" \
-H "Content-Type: application/json" \
-d '{"model": "vllm-inference/llama-3-2-3b", "messages":
[{"role": "user", "content": "Hello!"}], "max_tokens": 50}'
```

Expected result: HTTP **401**.

4.6. CONFIGURING ATTRIBUTE-BASED ACCESS CONTROL (ABAC) ON YOUR LLAMA STACK SERVER

Llama Stack supports OAuth 2.0/OIDC authentication with attribute-based access control (ABAC) for multi-tenant isolation. ABAC provides multi-tenant isolation by configuring access policies based on specific attributes, assigned to a user and the requested resource. When enabled, users can only access resources they own based on the attributes, and system resources are readable by all authenticated users.

The following procedure describes how to enable attribute-based access control (ABAC) policies in your Llama Stack distribution.

Prerequisites

- You have installed OpenShift 4.19 or later.
- You have logged in to Red Hat OpenShift AI.
- You have cluster administrator privileges for your OpenShift cluster.
- You have access to an OAuth 2.0/OIDC identity provider, for example, a Keycloak provider.

Procedure

1. The **AUTH_*** parameters also need to be set in the **LlamaStackDistribution** custom resource. For example:

Example LlamaStackDistribution CR

```
spec:
  replicas: 1
  server:
    containerSpec:
      env:
      ...
      - name: AUTH_ISSUER
        value: https://keycloak-redhat-ods-
applications.apps.rosa.derecluster.gm8d.p3.openshiftapps.com/realms/llamastack-demo
      - name: AUTH_JWKS_URI
        value: http://keycloak:8080/realms/llamastack-demo/protocol/openid-connect/certs
```

The **server.auth** section of the **config.yaml** file includes the authentication environment variables, these specifications uses OAuth2 token validation:

```
server:
  auth:
    provider_config:
      type: ${env.AUTH_ISSUER:+oauth2_token}
      audience: ${env.AUTH_AUDIENCE:=llama-stack}
      issuer: ${env.AUTH_ISSUER:=}
    jwks:
      uri: ${env.AUTH_JWKS_URI:=}
      key_recheck_period: ${env.AUTH_JWKS_RECHECK_PERIOD:=3600}
      verify_tls: ${env.AUTH_VERIFY_TLS:=true}
```

Table 4.1. Environment variables reference

Variable	Description	Default
AUTH_ISSUER	OpenID connect (OIDC) issuer URL. If unset, authentication is disabled	None
AUTH_AUDIENCE	Expected token audience	llama-stack

Variable	Description	Default
AUTH_JWKS_URI	JSON Web key set (JWKS) endpoint for token validation	None
AUTH_JWKS_RECHECK_PERIOD	How often, in seconds, to refresh JWKS keys	3600
AUTH_VERIFY_TLS	Verify TLS when fetching JWKS	true

- The client user must include a valid JWT bearer token in requests, for example:

```
$ curl -H "Authorization: Bearer <token>" \
https://llama-stack.example.com/v1/models
```

- The Llama Stack distribution ships with a default access policy:

```
access_policy:
- permit:
  actions: [read]
  when: resource is unowned
  description: "All users can read system resources"
- permit:
  actions: [create]
  description: "Authenticated users can create resources"
- permit:
  actions: [read, update, delete]
  when: user is owner
  description: "Owners can manage their own resources"
```

You can change these policies and create custom permissions for resource allocation.

The default policy describes the following behaviors for users:

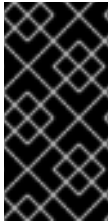
- **System resources are readable by all** - Resources without an owner. Models, shields, benchmarks registered in configuration are readable by any authenticated user.
- **Any authenticated user can create resources**- Users can create their own vector databases, files, datasets, conversations, etc.
- **Users can only manage their own resources**- Read, update, and delete operations on owned resources are restricted to the resource owner.

This access policy applies to user-created resources including: Vector databases, Files, Datasets, Conversations, Responses, Agents. While system resources registered in the **config.yaml** file do not have an owner and are accessible by all user types.

4.7. ABOUT USING SELF-SIGNED CERTIFICATES WITH LLAMA STACK

You can configure a **LlamaStackDistribution** custom resource (CR) to trust certificates that are issued by self-signed or private Certificate Authorities (CAs). This configuration enables the Llama Stack server to establish secure TLS connections to external inference, embedding, or vector store providers.

To configure a custom CA bundle, you reference a config map that contains the CA certificates from the **spec.server.tlsConfig.caBundle** field of the CR. The Llama Stack Operator validates the certificates, mounts a concatenated bundle into the Llama Stack server pod, and sets the **SSL_CERT_FILE** environment variable so that TLS clients in the server trust the bundle automatically.



IMPORTANT

When you configure or change the CA bundle for a **LlamaStackDistribution** CR, the Llama Stack Operator restarts the Llama Stack server pod so that the new certificates take effect. Plan for a brief service interruption when you apply or update the CA bundle on a **LlamaStackDistribution** CR that is serving production traffic.

For the procedure and the Llama Stack Operator processing details, see [Configuring a CA bundle for Llama Stack](#) in *Installing and uninstalling Red Hat OpenShift AI*.

4.8. ENABLING HIGH AVAILABILITY AND AUTOSCALING FOR LLAMA STACK

You can configure Llama Stack servers to remain available if a pod restarts, an application crashes, or node maintenance occurs. You can also enable autoscaling to adjust server capacity automatically based on resource usage. This procedure shows how to configure high availability and autoscaling for Llama Stack server pods by using the **LlamaStackDistribution** custom resource.

Prerequisites

- You have installed OpenShift 4.19 or later.
- You have logged in to Red Hat OpenShift AI.
- You have cluster administrator privileges for your OpenShift cluster.
- You have activated the Llama Stack Operator in OpenShift AI. For more information, see [Activating the Llama Stack Operator](#).
- You have installed the OpenShift CLI (**oc**) as described in the documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS

Procedure

1. To enable high availability for your Llama Stack server, add the following parameters to your **LlamaStackDistribution** CR:

```
spec:
  replicas: 2 1
  server:
    podDisruptionBudget:
      maxUnavailable: 1 2
```

```

topologySpreadConstraints: 3
- maxSkew: 1 4
  topologyKey: topology.kubernetes.io/zone 5
  whenUnsatisfiable: ScheduleAnyway 6
  labelSelector:
    matchLabels:
      app.kubernetes.io/instance: llamastackdistribution-sample 7

```

- 1 Runs two Llama Stack pods for high availability.
- 2 Specifies voluntary disruption tolerance. This configuration keeps at least one server pod available during voluntary disruptions.
- 3 Specifies how matching pods are spread across the cluster topology.
- 4 Instructs the scheduler to minimize replica imbalance across zones. With two replicas, the scheduler attempts to place one pod per zone.
- 5 Uses the node zone label as the failure domain for pod spreading.
- 6 Allows scheduling to proceed even if spread constraints cannot be fully satisfied.
- 7 Ensures that only pods from the same application instance are considered when calculating spread.

2. To enable autoscaling for your Llama Stack server, add the following parameters to your **LlamaStackDistribution** CR:

```

spec:
  server:
    autoscaling: 1
      minReplicas: 1 2
      maxReplicas: 5 3
      targetCPUUtilizationPercentage: 75 4
      targetMemoryUtilizationPercentage: 70 5

```

- 1 Configures a HorizontalPodAutoscaler (HPA) for the server pods.
- 2 Specifies the minimum number of replicas maintained by the HPA.
- 3 Specifies the maximum number of replicas maintained by the HPA.
- 4 Enables CPU-based scaling.
- 5 Enables memory-based scaling.

Additional information

- [Controlling pod placement by using pod topology spread constraints](#)
- [Automatically scaling pods with horizontal pod autoscaling](#)

